Ministério do Meio Ambiente

Cofre de Toxinas do MMA

Sistema de Controle de Acesso Biométrico

Documentação Técnica — Implementação, Lógica, Machine Learning / Deep Learning e Algoritmos

Aluno(a) _____

Curso _____

Professor(a) _____

Instituição _____

Data __ / __ / _____

Arquivo-fonte: `app.py` (~3.391 linhas) · Banco: `controle_acesso.db` (SQLite)

Sumário

1. Visão geral do sistema
2. Arquitetura e fluxo de funcionamento
3. Tecnologias e bibliotecas utilizadas
4. Modelo de dados (SQLite)
5. Explicação do código em blocos (do início ao fim)
6. Machine Learning / Deep Learning aplicados
7. Algoritmos relevantes implementados
8. Regras de segurança e controle de acesso
9. Como executar o sistema
10. Observações para apresentação

1. Visão geral do sistema

O sistema **Cofre de Toxinas do MMA** é uma aplicação web (desktop/mobile) construída em Python com a biblioteca **Streamlit**. Ele simula o controle de acesso físico e lógico a um cofre de alta segurança do Ministério do Meio Ambiente, onde estão guardadas toxinas perigosas.

O acesso é liberado por meio de **reconhecimento facial biométrico** e protegido por um **módulo anti-fraude (liveness)** que verifica se existe uma pessoa real diante da câmera — e não uma foto, vídeo ou tela de celular. Após a validação, o sistema cria uma **sessão** para o colaborador e libera os compartimentos do cofre conforme seu **nível de acesso cumulativo** (1 a 3).

Toda operação é registrada em um banco **SQLite**: cada acesso permitido ou negado, cada tentativa de fraude, cada erro da aplicação e cada configuração. Dashboards analíticos, relatórios em PDF e exportações CSV permitem auditoria completa.

Fluxo principal

1. O colaborador escolhe qual compartimento do cofre deseja acessar.
2. Captura a imagem (foto pela webcam) ou um **mini vídeo anti-fraude** (4 a 8 quadros).
3. O módulo de *liveness* analisa se é uma pessoa real (movimento, piscada, ausência de tela etc.).
4. O rosto do melhor quadro é comparado com todos os colaboradores cadastrados usando o modelo **VGG-Face** (similaridade por distância de cosseno).
5. Se batido, o nível do colaborador é comparado com o nível exigido do compartimento (permissões cumulativas).
6. Acesso permitido → abre a sessão e libera os compartimentos; negado → registra o evento, conta tentativas e pode acionar *bloqueio temporário*.

2. Arquitetura e fluxo de funcionamento

O projeto é de arquivo único (`app.py`), onde o *frontend* (interface gráfica) e o *backend* (lógica, visão computacional e banco) coexistem. O Streamlit executa o script de cima para baixo a cada interação e mantém o estado da sessão do usuário no `st.session_state`.

Componentes

Componente	Responsabilidade
Interface (Streamlit)	Radios de navegação, formulários de cadastro, câmera (`st.camera_input`), dashboards com Altair, botões e barras laterais.
Captura	Webcam integrada do navegador para foto ou até 8 quadros (mini vídeo).
Visão computacional	OpenCV (conversões, redimensionamento, FFT, componentes conectados), MTCNN (detecção de faces e landmarks), dlib (68 landmarks do rosto).
Deep Learning facial	DeepFace com backbone *VGG-Face*: embeddings e comparação por similaridade de cosseno.
Persistência	SQLite (`controle_acesso.db`) com 5 tabelas e controle de versão (`PRAGMA user_version`); fotos em `fotos_cadastradas/`.
Relatórios	PDF gerado com `fpdf2` (fonte Unicode DejaVu) e gráficos Altair convertidos para PNG com `vl-convert`.

Pipeline de validação biométrica

```
Câmera / mini vídeo
   |
   ▼
Captura de quadros (bytes JPEG + timestamp)
   |
   ▼
LIVENESS V2 (6 barreiras anti-fraude)
   |  - janela temporal ≥ 2 s
   |  - movimento suficiente (absdiff)
   |  - padrão orgânico (var. pixel + componentes conectados)
```

```
   |  - parallax 3D (variação geométrica nariz/olhos/boca)
   |  - piscada (EAR do dlib)
   |  - ausência de tela (FFT / padrão Moiré)
   ▼
Rosto detectado (MTCNN) → recorte do melhor quadro
   |
   ▼
RECONHECIMENTO (DeepFace / VGG-Face)
   |  - embeds de todos os colaboradores cadastrados
   |  - distância de cosseno (limiar ≤ 0.35)
   ▼
DECISÃO (_conceder_ou_negar)
   |  - nível do colaborador ≥ nível exigido? → PERMITIDO (abre sessão)
   |  - caso contrário ou desconhecido → NEGADO + auditoria
   ▼
Possível BLOQUEIO TEMPORÁRIO (3 tentativas em 10 min → 15 min bloqueio)
```

3. Tecnologias e bibliotecas utilizadas

Ambiente: **Python 3.12** em máquina virtual (venv), sistema Linux. A linha 4 desativa a GPU/CUDA (evita mensagens ruidosas no terminal).

3.1 Biblioteca padrão (Python)

Import	Função usada no código
`os`	Variáveis de ambiente, caminhos das fotos, verificação de arquivos.
`io`	Leitura de bytes de imagem com `Image.open(io.BytesIO(...))` e `pdf.image()`.

`hashlib`	SHA-1 dos quadros capturados — impede gravar quadros duplicados no mini vídeo.
`glob`	Localiza o arquivo `shape_predictor_68_face_landmarks.dat` do dlib dentro do pacote instalado.
`json`	Serializa os *detalhes* dos logs e das tentativas (JSON com acentuação).
`time`	Timestamps dos quadros e controle de expiração da sessão.
`datetime / timedelta`	Datas/horas nos registros, janela e duração do bloqueio.
`sqlite3`	Banco de dados local (DML/DDL).
`math`	Seno/tendência na série temporal simulada e aritmética no PDF.
`re` (interna)	Expressões regulares para extrair o número do nível em rótulos.

3.2 Bibliotecas de terceiros

Biblioteca	Versão (venv)	Propósito no projeto
streamlit	1.62.0	Interface web completa (inputs, câmera, métricas, gráficos, sidebar).
deepface	0.0.100	Orquestração do reconhecimento facial (VGG-Face) e comparação por cosseno.
opencv-python (cv2)	5.0.0.93	Controle da vision: decodificação de bytes, redimensionamento, `absdiff`, FFT, `connectedComponents`.
numpy	2.5.2	Vetores, normas euclidianas (EAR/parallax), FFT 2D, estatísticas (percentil, variância).
mtcnn	1.0.0	Detecção de rostos e keypoints (olhos, nariz, boca) para o liveness.
dlib	20.0.	Detector frontal + preditor de 68 landmarks

	1	(cálculo do EAR da piscada).
face-recognition / face_recognition_models	1.3.0 / 0.3.0	Fornecem o binário `shape_predictor_68_face_landmarks.dat` usado pelo dlib (localizado via glob).
pillow (PIL)	12.3.0	Abertura, recorte e redimensionamento de imagens (thumbnails, recorte do rosto).
pandas	3.0.5	DataFrames para dashboards, agregações e exportações CSV.
altair	6.2.2	Gráficos interativos dos dashboards e do relatório PDF.
vl-convert-python	1.9.0.post1	Renderiza gráficos Altair em PNG (memória) para o relatório PDF.
fpdf2 (fpdf.FPDF)	2.8.8	Geração do relatório executivo em PDF com fonte Unicode DejaVu.
openpyxl (auxiliar)	3.1.5	Gera a planilha de exemplo `Relatorio_Controle_Acesso_Facial.xlsx` (script `gerar_excel.py`).

Observação: `geopandas` e `retina-face` estão instalados como dependências do ecossistema, mas o fluxo principal usa `detector_backend="skip"` (a detecção é feita pelo MTCNN próprio) e o mapa GeoPandas não é chamado no app.

4. Modelo de dados (SQLite)

Banco `controle_acesso.db` com versão atual **3** (`PRAGMA user_version`). As tabelas são criadas em `init_db()` e evoluem via `migrar_banco_legado()`.

Tabela	Colunas	Finalidade
colaboradores	id, nome, cargo, divisao, nivel_acesso (1-3), foto_path, data_cadastro	Dados e nível de acesso de cada colaborador; referência de foto para o reconhecimento.
logs_acesso	id, colaborador_nome, setor,	Auditoria de todos os eventos de

	data_hora, status (PERMITIDO/NEGADO), detalhes (JSON)	acesso com diagnóstico em JSON (motivo do liveness, distâncias etc.).
logs_erros	id, data_hora, tipo, modulo, mensagem, descricao, sugerida	Registro automático de erros com ajuda amigável para o operador.
tentativas_seguranca	id, compartimento, data_hora, motivo, detalhes (JSON), origem	Histórico de tentativas de fraude/negação por compartimento — base do bloqueio temporário.
configuracoes	chave (PK), valor	Preferências persistentes do sistema (ex.: posições das máscaras da home).

Migrações (user_version)

- **v0 → v1:** adiciona `nivel_acesso` (e `divisao`) e reinicia colaboradores legados para Nível 1.
- **v1 → v2:** adiciona a coluna `detalhes` (JSON) em `logs_acesso`.
- **v2 → v3:** cria a tabela `configuracoes` para persistir as preferências do sistema.

5. Explicação do código em blocos (do início ao fim)

A seguir, cada bloco do `app.py` é explicado na ordem em que aparece no arquivo, com a linha de referência, o que faz e o propósito. O código-fonte integral está no **Anexo A** (e o script auxiliar `gerar_excel.py` no **Anexo B**).

Bloco 1 — Importações e desativação da GPU (linhas 1-25)

Importa todas as bibliotecas listadas na seção 3. A instrução `os.environ["CUDA_VISIBLE_DEVICES"] = "-1"` no início (antes do TensorFlow/DeepFace) impede que o TensorFlow tente usar GPU, evitando avisos e erros de driver em máquinas sem CUDA.

Bloco 2 — Configuração da página e CSS da moldura (linhas 27-62)

`st.set_page_config` define título, ícone e layout largo. A constante `MOLDURA_CIRCULAR_CSS`

injeta CSS que desenha um círculo verde tracejado sobre a câmera (`stCameraInput`) — orienta o usuário a centralizar o rosto. `DB_FILE` e `FOTOS_DIR` definem o banco e a pasta de fotos, criada se não existir.

Bloco 3 — Banco de dados e criação das tabelas (linhas 65-127)

`init_db()` usa `CREATE TABLE IF NOT EXISTS` para garantir as 5 tabelas (seção 4). Funciona como inicialização idempotente.

Bloco 4 — Migração do banco legado (linhas 129-176)

`migrar_banco_legado()` adapta bancos criados pelas versões anteriores do sistema (permissões por setor e níveis 1-4). Primeiro consulta as colunas com `PRAGMA table_info`; adiciona as colunas que faltam; usa `PRAGMA user_version` para executar cada migração uma única vez e preenche divisões vazias com "Sem divisão".

Bloco 5 — Configurações persistentes (linhas 179-267)

A tabela `configuracoes` (chave, valor) é usada para persistir preferências do sistema, como as posições das máscaras clicáveis da home. As funções de leitura e gravação usam `ON CONFLICT DO UPDATE` e são carregadas na inicialização do script.

Bloco 6 — Níveis de acesso e rótulos (linhas 274-316)

`NIVEIS_ACESSO` define os três níveis cumulativos do cofre: 1. *Permissão Geral*, 2. *Diretor de Divisões Específicas*, 3. *Ministro do Meio Ambiente*. Os helpers `nome_nivel`, `descricao_nivel` e `rotulo_nivel` convertem o número em texto para a interface (ex.: "Nível 2 – Diretor de Divisões Específicas").

Bloco 7 — Normalização de compartimentos legados (linhas 319-362)

`COMPARTIMENTOS_LEGADOS` mapeia rótulos antigos (níveis 1-4 e setores pré-níveis) para os três níveis atuais. `normalizar_compartimento` aplica o mapa ou usa regex (`Nível\s*\d`) para inferir o nível em logs antigos; `rotulo_curto_compartimento` remove o prefixo "Nível N – " para exibição compacta.

Bloco 8 — Dados simulados do cofre (linhas 365-552)

`DADOS_TOXINAS` é o inventário (simulado) com 10 toxinas (Ricina, Tetrodotoxina, Toxina

Botulínica, Aflatoxina etc.), cada uma com periculosidade, estoque, temperatura, validade e estado de conservação. `DIVISOES_MMA` relaciona as divisões do ministério. Dali derivam: `carregar_dados_toxinas()` (DataFrame pandas) e `serie_temporal_movimentacao()`, que gera uma série mensal simulada usando seno (sazonalidade) + tendência linear — função `int((idx+1)*8 + sazonalidade + tendencia)`.

Bloco 9 — Sessão e permissões (linhas 555-576)

`LIMITE_INATIVIDADE_S = 300` (5 min). `_sessao_valida()` confere `usuario_logado` e o timestamp do login; se a inatividade exceder 5 minutos, remove a sessão e marca `sessao_expirada`. `usuario_tem_nivel(n)` retorna True se o usuário logado tem nível maior ou igual a n (permissão cumulativa).

Bloco 10 — Preparação dos logs para dashboards (linhas 579-598)

`_preparar_logs()` carrega `logs_acesso`, renomeia a coluna `setor` para `area_dados`, normaliza os rótulos legados, extrai o `nivel` por regex, além de criar `compartimento_curto`, `data` e `hora`.

Bloco 11 — Relatório PDF executivo (linhas 601-1112)

Conjunto de helpers sobre `fpdf2`:

- `_chart_png()` — converte um gráfico Altair em PNG em memória via `vl-convert`;
- `_RelatorioPDF` — subclasse que adiciona rodapé com número da página;
- `_registrar_fonte_pdf()` — registra a fonte Unicode *DejaVu Sans* (necessária para acentos e travessões);
- `_secao_pdf()`, `_inserir_kpi()`, `_row_kpis()` — barras de seção e cartões de indicadores;
- `_inserir_grafico()` — insere o PNG com legenda, com quebra de página automática;
- `_tabela_pdf()` — tabelas com cabeçalho escuro, zebra e destaque de coluna (verde/vermelho para status).

`gerar_relatorio_pdf(df_filtrado)` monta o documento: cabeçalho institucional, indicadores, 4 gráficos de acesso (tendência diária, distribuição por nível, horários, top colaboradores), panorama do cofre (estoque e série temporal), logs de acesso, equipe, catálogo de toxinas e cadeia de custódia.

Bloco 12 — Recorte do centro do rosto (linhas 1115-1128)

`recortar_centro_rosto()` usa PIL para cortar a região central da imagem (20%→80% em x, 5%→85% em y), concentrando o processamento no rosto e removendo ruído de fundo.

Bloco 13 — Detecção de faces com MTCNN (linhas 1130-1159)

`_detector_mtcnn()` carrega o detector MTCNN apenas na primeira chamada (import lazy). `_imgs_bytes_para_numpy()` decodifica os bytes JPEG para arrays RGB com OpenCV. `_regiao_uniao()` retorna a caixa que engloba duas caixas de rosto (usada para medir movimento na região comum). *Por que MTCNN e não o Haar Cascade do OpenCV?* — as versões novas do OpenCV removeram o `CascadeClassifier`, e o MTCNN também fornece os keypoints (olhos/nariz/boca) necessários ao parallax.

Bloco 14 — Liveness V2: anti-fraude (linhas 1162-1473)

É o coração do sistema. Constantes de limiar (seção 6.2). A função `avaliar_liveness(frames)` recebe a lista de `(bytes, timestamp)` e retorna `(aprovado, motivo, detalhes, idx_melhor_quadro)`, aplicando as 6 barreiras em cadeia. As funções auxiliares são: `_carregar_preditor_dlib()` (localiza o modelo de 68 landmarks), `_ear_dos_olhos()` (EAR), `_piscada_detectada()`, `_variacao_paralaxe()` e `_detectar_tela()` (FFT). Detalhes na seção 6.

Bloco 15 — Reconhecimento biométrico (linhas 1476-1506)

`_melhor_match_biometrico(temp_test_path)` percorre todos os cadastrados e chama `DeepFace.verify(img1, img2, model_name="VGG-Face", detector_backend="skip", distance_metric="cosine", enforce_detection=False)`, guardando o candidato de menor distância. O `detector_backend="skip"` evita detecção duplicada (o rosto já foi recortado), e `enforce_detection=False` evita exceção em imagem difícil. Limiar de aceite: **distância ≤ 0.35**.

Bloco 16 — Decisão de acesso (linhas 1509-1564)

`_conceder_ou_negar(...)` busca o nível real do colaborador no banco. Se `nivel_colab ≥ nivel_requerido`: sucesso, log PERMITIDO, cria `usuario_logado` na sessão (com `login_ts`) e toast. Senão: log NEGADO com motivo "Nível insuficiente", registra tentativa e causa de bloqueio.

Bloco 17 — Política de bloqueio temporário (linhas 1567-1671)

Constantes: `JANELA_FRAUDE_S=600` (10 min, janela de contagem), `DURACAO_BLOQUEIO_S=900` (15 min de bloqueio) e `MAX_TENTATIVAS=3`. `_registrar_tentativa_fraude()` insere na tabela e conta quantas ocorreram na janela; `_bloqueio_ativo()` calcula (bloqueado, minutos restantes); `_tentativas_recentes()` lista as últimas para a tela; `_registrar_e_verificar_bloqueio()` registra, verifica e aciona o bloqueio do compartimento (controlado por `alertou_bloqueio`).

Bloco 18 — Mini vídeo anti-fraude (linhas 1674-1816)

`_render_mini_video()` é a UI: `st.camera_input` captura até 8 quadros; a deduplicação usa SHA-1 dos bytes; métrica "Quadros capturados", botão de limpar, miniaturas e o botão "Analisar Mini Vídeo e Validar Acesso" (desabilitado abaixo de 3 quadros). Na análise: roda o liveness; se negado, loga, registra tentativa e (se atingiu o limite) bloqueia; se aprovado, usa o melhor quadro para o reconhecimento VGG-Face e a decisão final. Todo o fluxo tem *try/except* com registro de erro.

Bloco 19 — Registro de logs (linhas 1819-1836)

`registrar_log(nome, area, status, detalhes=None)` insere em `logs_acesso` com timestamp e serializa os detalhes em JSON.

Bloco 20 — Erros e ajuda ao operador (linhas 1839-1960)

`obter_ajuda_erro(tipo, mensagem)` devolve (descrição, sugestão) amigáveis com base em padrões de texto (ex.: imagem corrompida, biometria sem rosto, banco travado, timeout). `registrar_erro()` grava na `logs_erros`; `limpar_logs_erros()` apaga o histórico.

Bloco 21 — Persistência de preferências (linhas 1963-2138)

Funções de leitura e gravação de preferências na tabela `configuracoes` (chave, valor), com `ON CONFLICT DO UPDATE`, usadas para guardar as posições das máscaras clicáveis da home e outras opções do sistema. As preferências são recarregadas na inicialização de cada sessão.

Bloco 22 — Cadastro de colaboradores (linhas 2141-2203)

`salvar_colaborador()` valida nome e foto, grava o arquivo JPEG em `fotos_cadastradas/`

(nome sanitizado + timestamp), recorta o rosto, insere a linha em `colaboradores` e limpa o formulário. `carregar_modelo_facial()`, com `@st.cache_resource`, carrega o modelo VGG-Face uma única vez para todas as validações.

Bloco 23 — Sidebar: navegação e sessão (linhas 2206-2303)

Radio de navegação com as 6 telas. Mostra o usuário logado e o nível (ou aviso de sessão expirada) com botão "Encerrar Sessão".

Bloco 24 — Abas da aplicação (linhas 2317-3391)

Cada aba é envolvida por `ao_trocar_aba_guard()`, que captura erros inesperados e os registra em `logs_erros`:

- 📸 **Cadastro de Colaboradores:** dados pessoais, divisão, nível (cumulativo, com descrição), captura por webcam *ou upload* (o upload foi preservado aqui; a validação aceita apenas webcam em gravação de imagem que é analisada como proprietária). Lista os cadastrados.
- 📹 **Validar Acesso / Login Biométrico:** escolhe o compartimento (nível 1-3); verifica bloqueio ativo (exibe motivos e data das tentativas e encerra); radio entre "Tirar Foto pela Webcam" e "🎥 Mini Vídeo Anti-Fraude". No modo foto, chama recorte + modelo + `_melhor_match_biometrico` + `_conceder_ou_negar`.
- 📈 **Dashboard Analítico de Acessos:** filtros de período, compartimento, status e nome; métricas de total/permitidos/negados/taxa/média; gráficos de tendência temporal, pizza de status, distribuição por compartimento (níveis cumulativos), equipe, top colaboradores, acessos por hora e o "Diagnóstico de Segurança" (compartimentos mais visados, pessoas desconhecidas e colaboradores sem permissão); exportação CSV e relatório PDF filtrado.
- 📊 **Dashboard e Relatórios:** visão consolidada (resumo, filtros, histórico, CSV e PDF completo).
- 🐞 **Registro de Erros / Logs:** filtros por tipo/módulo/ mensagem/data, resumo com gráfico de barras, lista, e expanção Explicação/Sugestão/Detalhe técnico para cada erro, exportação CSV e limpeza.
- 🔐 **Cofre de Toxinas:** tela principal protegida por sessão. Nível 1: catálogo público + missão da sala; Nível 2: inventário e localização (gráficos Altair + estoque/conservação + CSV); Nível 3: dossiê completo, série temporal de movimentações por divisão e cadeia de custódia; Governança (nível 3): métricas,

colaboradores, indicadores de estado crítico e base completa.

6. Machine Learning / Deep Learning aplicados

O sistema combina **aprendizado profundo** (reconhecimento facial) com **visão computacional clássica e estatística** (o anti-spoofing).

6.1 Reconhecimento facial — DeepFace / VGG-Face

O reconhecimento usa o **DeepFace** como orquestrador e **VGG-Face** como modelo de extração de características (*embeddings*). O VGG-Face é uma rede convolucional profunda treinada na classificação de milhares de identidades; criou-se um espaço de características onde rostos do mesmo indivíduo ficam próximos.

```
DeepFace.verify(
    img1_path=temp_test_path,     # rosto capturado (recortado)
    img2_path=cpath,              # foto cadastrada
    model_name="VGG-Face",        # backbone de embeddings
    detector_backend="skip",      # já detectamos com o MTCNN
    distance_metric="cosine",     # similaridade por cosseno
    enforce_detection=False,      # não lança erro sem rosto detectável
)
```

A rede transforma cada imagem em um vetor (embedding). A **similaridade** entre dois rostos é medida pela **distância de cosseno**, que varia de 0 (idênticos) a 1 (opostos). O sistema aceita o colaborador quando a menor distância entre a captura e os cadastrados é **≤ 0.35** — valor padrão recomendado da biblioteca. O modelo é carregado uma única vez com `@st.cache_resource` (`DeepFace.build_model`) para desempenho.

6.2 Liveness — as 6 barreiras anti-fraude (V2)

Uma foto impressa, um vídeo pré-gravado ou uma foto exibida na tela de um celular têm características que as distinguem de um rosto real. O módulo usa seis barreiras

independentes; basta uma falhar para **negar**.

#	Barreira	Limiar	Decorre de
1	Mínimo de quadros e janela temporal	≥ 3 quadros e ≥ 2,0 s	Fotos/rajadas são rápidas demais; um rosto real exige tempo para mover a cabeça e piscar.
2	Movimento na região facial	absdiff médio ≥ 0,015 (1,5%)	Diferença pixel a pixel entre quadros consecutivos na união das caixas; imagem parada falha.
3	Padrão orgânico de movimento	≥ 4 componentes conectados	Mapa de variância temporal dos pixels do rosto (percentil 75 como limite dinâmico) + `connectedComponents` de 8-vizinhança; rosto real varia em várias regiões (olhos, boca, bochechas), foto desliza como um bloco único.
4	Parallax 3D	std ≥ 0,05	Proporções nariz→olhos→boca normalizadas pela distância interocular; girar a cabeça altera essas proporções (3D); uma foto plana só translada (2D) e mantém proporções constantes.
5	Piscada	EAR mínimo < 60% do baseline	`Eye Aspect Ratio` via 68 landmarks do dlib; fotos e telas não piscam. Baseline é o valor máximo (olho aberto) da sequência.
6	Ausência de tela (Moiré)	pico/média na alta frequência < 12,0	FFT 2D do recorte do rosto (com janela de Hann); telas produzem grades periódicas de alta frequência (padrão Moiré) — o anel de alta frequência destaca-se.

Cada barreira acumula `detalhes` diagnósticos (movimento %, regiões de variação, parallax, piscada, sinal de tela) que são gravados no JSON do log para auditoria e para mensagens explicativas ao usuário. As justificativas educam o operador ("as fotos nunca piscam", "foto deslizando não tem parallax").

6.3 Por que essas técnicas?

- **Fotos impressas:** falham nas barreiras 2, 3, 4 e 5 (sem movimento, rigidez, sem

piscada).

- **Fotos na tela do celular:** adicionalmente falham na barreira 6 (Moiré da tela).
- **Vídeos pré-gravados passados à câmera:** tendem a ter janela curta, movimento rígido ou sinal de tela — e, se o vídeo não contiver piscada, reprovam na barreira 5.
- **O custo de cada tentativa** (contagem de tentativas com bloqueio após 3 em 10 min) inviabiliza ataques por força bruta.

7. Algoritmos relevantes implementados

7.1 Distância de cosseno (recognition)

Dados dois vetores de características A e B:

```
cos(A,B) = (A · B) / (||A|| · ||B||)      →  distancia = 1 - cos
```

Implementada pelo DeepFace. Varre o cadastro em `_melhor_match_biometrico` guardando o candidato de menor distância.

7.2 Eye Aspect Ratio — EAR (piscada)

Com os 6 pontos de contorno de cada olho (índices 36-41 e 42-47 dos 68 landmarks do dlib):

```
EAR = (|P2-P6| + |P3-P5|) / (2 · |P1-P4|)
```

Olho aberto ≈ 0,25-0,33; piscado muito menor. O código usa `np.linalg.norm` para as distâncias euclidianas. Detecta-se piscada quando o mínimo da série **cai abaixo de 60%** do baseline aberto (`LIMIAR_PISCADA=0.60`).

7.3 FFT 2D + janela de Hann (detecção de tela)

No recorte do rosto, `_detectar_tela`:

1. Redimensiona o rosto para 256×256;
2. Multiplica pela janela de Hann 2D (`np.outer(np.hanning(256), np.hanning(256))`) — reduz o vazamento espectral das bordas;

3. Aplica `np.fft.fft2` e desloca o espectro com `fftshift`;
4. Mascara um **anel de alta frequência** (raio entre 55% e 47% da máxima distância ao centro);
5. Calcula `máximo / média` nesse anel. Grades de tela concentram energia em picos periódicos → valor alto (≥ 12) indica tela.

7.4 Componentes conectados (organicidade)

A variância pixel a pixel dos rostos alinhados (80×80) gera uma *máscara* das regiões que mais mudaram (≥ percentil 75). Com `cv2.connectedComponents(connectivity=8)` conta-se quantas regiões independentes existem. Um rosto real move olhos, bochechas e boca de forma independente (≥ 4 regiões); uma foto que desliza move tudo junto (1-3 regiões).

7.5 Parallax geométrico

Para cada quadro, usando os keypoints do MTCNN calcula-se: `offset_x = (nariz.x - centroOlhos.x) / larguraOlhos` e `offsetY_boca = (boca.y - centroOlhos.y) / larguraOlhos`. O desvio padrão dessas proporções na sequência é o "parallax". A normalização pelo espaço interocular torna a medida invariante à distância da câmera.

7.6 Diferença absoluta temporal (movimento)

`cv2.absdiff(quadro_a, quadro_b)` na região união das caixas de rosto, normalizado por 255, média global — mede o quanto a cena mudou entre quadros consecutivos.

7.7 Geração de dados simulados (séries)

Sazonalidade senoidal (`sin(i/6·π)`) + tendência linear (`(i/n)·4`) para compor as movimentações mensais; distribuição dos estoques do inventário manual (DADOS_TOXINAS).

8. Regras de segurança e controle de acesso

8.1 Níveis cumulativos

Nível **1** → catálogo público; Nível **2** → inventário/localização/movimentação; Nível **3** → dossiê completo, cadeia de custódia e governança. Um colaborador de nível N enxerga

todos os módulos dos níveis 1..N (`nivel_acesso >= requerido`).

8.2 Bloqueio temporário por compartimento

- 3 tentativas negadas/de fraude na janela de 10 min (`JANELA_FRAUDE_S=600`) → bloqueio de **15 min** (`DURACAO_BLOQUEIO_S=900`).
- O bloqueio é por compartimento (`area_requerida`); a tela de validação encerra e mostra as tentativas recentes com data/hora, motivo e origem.
- O bloqueio é acionado uma única vez por sessão (controle `alertou_bloqueio` no session_state).

8.3 Sessão por inatividade

Após **5 minutos** sem interação, a sessão expira e o cofre fica inacessível até novo login biométrico (`_sessao_valida`).

8.4 Auditoria

- `logs_acesso`: todo evento com status e diagnóstico JSON;
- `tentativas_seguranca`: histórico de fraudes por compartimento;
- `logs_erros`: erros com sugestão de resolução;
- Dashboards com diagnóstico de segurança (compartimentos mais visados, desconhecidos, colaboradores sem permissão).

8.5 Persistência de preferências

- Preferências simples são guardadas na tabela `configuracoes` (chave, valor), como as posições das máscaras clicáveis da home;
- As funções de leitura e gravação usam `ON CONFLICT DO UPDATE` e são recarregadas na inicialização de cada sessão.

9. Como executar o sistema

```
# 1) Criar/ativar o ambiente virtual

python3.12 -m venv venv

source venv/bin/activate
```

```
# 2) Instalar as dependências

pip install streamlit deepface opencv-python pillow numpy pandas altair             vl-
convert-python fpdf2 dlib mtcnn face-recognition



# 3) Executar

./venv/bin/streamlit run app.py --server.port 8502



# (opcional) gerar a planilha de exemplo

./venv/bin/python gerar_excel.py
```

Abrir `http://localhost:8502`. O banco e a pasta `fotos_cadastradas/` são criados automaticamente na primeira execução.

10. Observações para apresentação

- Demonstrar primeiro o **cadastro** de um colaborador (com foto), depois a **validação biométrica**.
- Mostrar o **mini vídeo anti-fraude** e tentar uma foto exibida na tela do celular — será bloqueada pelas barreiras (movimento/parallax/ piscada/Moiré).
- Abrir o **Dashboard Analítico** e o **Registro de Erros** para evidenciar a auditoria automática.
- Explicar que o bloqueio temporário protege contra força bruta, e que a expiração de sessão protege o cofre em caso de abandono do terminal.

Anexo A — Código-fonte completo: app.py

Reprodução integral do arquivo `app.py` (ambiente na data da documentação). As referências de linha citadas na seção 5 correspondem a este arquivo.

import os # Desativa avisos de GPU/CUDA no terminal os.environ["CUDA_VISIBLE_DEVICES"] = "-1" import io import hashlib import base64 import glob import json import secrets import time from datetime import datetime,

timedelta import sqlite3 from PIL import Image import cv2 import numpy as np from deepface import DeepFace import pandas as pd import streamlit as st import altair as alt import math from fpdf import FPDF st.set_page_config(page_title="Cofre de Toxinas — Ministério do Meio Ambiente", page_icon="🔐", layout="wide",) # CSS responsivo para dispositivos móveis MOBILE_CSS = """ <style> @media (max-width: 768px) { div[data-testid="stHorizontalBlock"] { flex-direction: column; } div[data-testid="stHorizontalBlock"] > div { width: 100% !important; flex: unset !important; min-width: 0 !important; } div[data-testid="stSidebar"] { width: min(85vw, 280px) !important; } div[data-testid="stMetricValue"] { font-size: 1.5rem !important; } h1 { font-size: 1.6rem ! important; } h2 { font-size: 1.3rem !important; } .stButton > button { width: 100% ! important; } div[data-testid="stCameraInput"] { max-width: 100% !important; } div[data-testid="stDataFrame"] { font-size: 0.8rem !important; } } </style> """ st.markdown(MOBILE_CSS, unsafe_allow_html=True) # Moldura circular visual no CSS MOLDURA_CIRCULAR_CSS = """ <style> div[data-testid="stCameraInput"] { position: relative; max-width: 500px; margin: 0 auto; } div[data-testid="stCameraInput"]::after { content: ""; position: absolute; top: 45%; left: 50%; transform: translate(-50%, -50%); width: 250px; height: 300px; border: 3px dashed #00FF66; border-radius: 50%; pointer-events: none; box-shadow: 0 0 0 9999px rgba(0, 0, 0, 0.4); z-index: 10; } </style> """ IMAGEMHOME_DIR = "IMAGEMHOME" EXTENSOES_IMAGEM = (".png", ".jpg", ".jpeg", ".webp") HOME_CSS = """ <style> div[data-testid="stImage"] img { border-radius: 14px; box-shadow: 0 6px 22px rgba(0, 0, 0, 0.35); } </style> """ DB_FILE = "controle_acesso.db" FOTOS_DIR = "fotos_cadastradas" if not os.path.exists(FOTOS_DIR): os.makedirs(FOTOS_DIR) def init_db(): conn = sqlite3.connect(DB_FILE) cursor = conn.cursor() cursor.execute(""" CREATE TABLE IF NOT EXISTS colaboradores (id INTEGER PRIMARY KEY AUTOINCREMENT, nome TEXT NOT NULL, cargo TEXT, divisao TEXT, nivel_acesso INTEGER NOT NULL DEFAULT 1, foto_path TEXT NOT NULL, data_cadastro TEXT) """) cursor.execute(""" CREATE TABLE IF NOT EXISTS logs_acesso (id INTEGER PRIMARY KEY AUTOINCREMENT, colaborador_nome TEXT, setor TEXT, data_hora TEXT, status TEXT) """) cursor.execute(""" CREATE TABLE IF NOT EXISTS logs_erros (id INTEGER PRIMARY KEY AUTOINCREMENT, data_hora TEXT, tipo TEXT, modulo TEXT, mensagem TEXT, descricao TEXT, sugerida TEXT) """) cursor.execute(""" CREATE TABLE IF NOT EXISTS tentativas_seguranca (id INTEGER PRIMARY KEY AUTOINCREMENT, compartimento TEXT, data_hora TEXT, motivo TEXT, detalhes TEXT, origem TEXT) """) cursor.execute(""" CREATE TABLE IF NOT EXISTS configuracoes (chave TEXT PRIMARY KEY, valor TEXT) """) conn.commit() conn.close() def migrar_banco_legado(): """Migra o banco antigo (permissões por setor / níveis 1-4) para o

novo esquema do Cofre de Toxinas: níveis cumulativos 1-3 e coluna de divisão.""" try: conn = sqlite3.connect(DB_FILE) cursor = conn.cursor() colunas = [r[1] for r in cursor.execute("PRAGMA table_info(colaboradores)")] if "nivel_acesso" not in colunas: cursor.execute("ALTER TABLE colaboradores ADD COLUMN nivel_acesso INTEGER NOT NULL DEFAULT 1") if "divisao" not in colunas: cursor.execute("ALTER TABLE colaboradores ADD COLUMN divisao TEXT") # Novo esquema de segurança: os colaboradores existentes do banco # antigo (permissões por setor / níveis 1-4) são reiniciados para o # Nível 1 (Permissão Geral) apenas na primeira execução da migração. cursor.execute("PRAGMA user_version") versao = cursor.fetchone()[0] if versao < 1: cursor.execute("UPDATE colaboradores SET nivel_acesso = 1") cursor.execute("PRAGMA user_version = 1") if versao < 2: cols_logs = [r[1] for r in cursor.execute("PRAGMA table_info(logs_acesso)")] if "detalhes" not in cols_logs: cursor.execute("ALTER TABLE logs_acesso ADD COLUMN detalhes TEXT") cursor.execute("PRAGMA user_version = 2") if versao < 3: cursor.execute(""" CREATE TABLE IF NOT EXISTS configuracoes (chave TEXT PRIMARY KEY, valor TEXT) """) cursor.execute("PRAGMA user_version = 3") if versao < 4: # Esquema de autenticação por credenciais (usuário/senha) e # controle de quem pode cadastrar novos colaboradores. if "usuario" not in colunas: cursor.execute("ALTER TABLE colaboradores ADD COLUMN usuario TEXT") if "senha_hash" not in colunas: cursor.execute("ALTER TABLE colaboradores ADD COLUMN senha_hash TEXT") if "pode_cadastrar" not in colunas: cursor.execute("ALTER TABLE colaboradores ADD COLUMN pode_cadastrar INTEGER NOT NULL DEFAULT 0") cursor.execute("CREATE UNIQUE INDEX IF NOT EXISTS idx_colaboradores_usuario " "ON colaboradores(usuario) WHERE usuario IS NOT NULL") cursor.execute("PRAGMA user_version = 4") cursor.execute("UPDATE colaboradores SET divisao = 'Sem divisão' " "WHERE divisao IS NULL OR divisao = '") conn.commit() conn.close() except Exception: pass def hash_senha(senha): """Gera o hash da senha com salt único (PBKDF2-HMAC-SHA256)."""" salt = secrets.token_hex(16) digest = hashlib.pbkdf2_hmac("sha256", senha.encode("utf-8"), bytes.fromhex(salt), 100_000) return f"{salt}${digest.hex()}" def verificar_senha(senha, armazenado): """Confere a senha contra o valor armazenado no formato 'salt$hash'."""" if not armazenado or "$" not in armazenado: return False try: salt, digest_hex = armazenado.split("$", 1) digest = hashlib.pbkdf2_hmac("sha256", senha.encode("utf-8"), bytes.fromhex(salt), 100_000) return secrets.compare_digest(digest.hex(), digest_hex) except Exception: return False NIVEIS_ACESSO = { 1: { "nome": "Permissão Geral", "descricao": ("Acesso concedido a indivíduos com permissão geral dentro do " "Ministério do Meio Ambiente. Libera o catálogo público e as " "informações gerais sobre o cofre de toxinas."), }, 2: { "nome":

"Diretor de Divisões Específicas", "descricao": ("Acesso restrito aos diretores de divisões específicas do " "Ministério. Libera o inventário, a localização e a movimentação " "das toxinas dos compartimentos sob sua responsabilidade."), }, 3: { "nome": "Ministro do Meio Ambiente", "descricao": ("Acesso exclusivo ao ministro do Meio Ambiente. Controle total " "sobre o cofre: dossiê completo, cadeia de custódia, gestão de " "colaboradores e auditoria."), }, } def nome_nivel(nivel): """Retorna o nome do nível de acesso.""" info = NIVEIS_ACESSO.get(nivel) return info["nome"] if info else f"Nível {nivel}" def descricao_nivel(nivel): """Retorna a descrição do nível de acesso.""" info = NIVEIS_ACESSO.get(nivel) return info["descricao"] if info else "" def rotulo_nivel(nivel): """Ex.: 'Nível 2 – Consulta Cartográfica'.""" return f"Nível {nivel} – {nome_nivel(nivel)}" # Mapeia rótulos legados (níveis antigos e setores) para os 3 níveis # atuais do Cofre de Toxinas apenas para exibição nos dashboards. COMPARTIMENTOS_LEGADOS = { # Níveis antigos (esquema 1-4 de dados geoespaciais) "Nível 1 – Consulta Básica de Imagens": rotulo_nivel(1), "Nível 2 – Consulta Cartográfica": rotulo_nivel(2), "Nível 3 – Análise Especializada": rotulo_nivel(3), "Nível 4 – Governança e Administração de Dados": rotulo_nivel(3), # Setores antigos (pré-níveis) "Diretoria Executiva": rotulo_nivel(3), "P&D / Laboratório": rotulo_nivel(3), "Tesouraria / Cofre": rotulo_nivel(3), "TI / Servidores": rotulo_nivel(1), "Recursos Humanos": rotulo_nivel(1), } def normalizar_compartimento(rotulo): """Converte rótulos legados de `setor` para o compartimento atual do cofre.""" if not isinstance(rotulo, str): return rotulo_nivel(1) rotulo = rotulo.strip() if rotulo in COMPARTIMENTOS_LEGADOS: return COMPARTIMENTOS_LEGADOS[rotulo] import re m = re.search(r"Nível\s*(\d)", rotulo) if m: nivel = int(m.group(1)) if nivel >= 3: return rotulo_nivel(3) if nivel == 2: return rotulo_nivel(2) return rotulo_nivel(1) return rotulo_nivel(1) def rotulo_curto_compartimento(rotulo): """Ex.: 'Nível 2 – Diretor de Divisões Específicas' -> 'Diretor de Divisões Específicas'.""" if isinstance(rotulo, str): import re return re.sub(r"^Nível\s*\d+\s*–\s*", "", rotulo) return str(rotulo) DADOS_TOXINAS = [{ "id_toxina": "TOX-0001", "nome": "Ricina", "classe": "Proteína", "tipo_acao": "Citotóxica", "periculosidade": 5, "dose_letal_mg": 0.002, "uso": "Pesquisa / neutralização", "compartimento": "Compartimento A-01", "divisao_responsavel": "Divisão de Toxicologia Ambiental", "estoque_unidades": 12, "data_validade": "2027-03-15", "temperatura_c": -20, "estado_conservacao": "Estável", }, { "id_toxina": "TOX-0002", "nome": "Saxitoxina", "classe": "Biotoxina marinha", "tipo_acao": "Neurotóxica", "periculosidade": 5, "dose_letal_mg": 0.20, "uso": "Controle de efluentes", "compartimento": "Compartimento A-02", "divisao_responsavel": "Divisão de Emergências Químicas", "estoque_unidades": 8, "data_validade": "2026-11-30", "temperatura_c": 4, "estado_conservacao": "Estável", },

{ "id_toxina": "TOX-0003", "nome": "Tetrodotoxina", "classe": "Biotoxina marinha", "tipo_acao": "Neurotóxica", "periculosidade": 5, "dose_letal_mg": 0.05, "uso": "Estudos toxicológicos", "compartimento": "Compartimento A-03", "divisao_responsavel": "Divisão de Pesquisa e Laboratório", "estoque_unidades": 5, "data_validade": "2026-09-12", "temperatura_c": -30, "estado_conservacao": "Atenção", }, { "id_toxina": "TOX-0004", "nome": "Microcistina-LR", "classe": "Cianotoxina", "tipo_acao": "Hepatotóxica", "periculosidade": 3, "dose_letal_mg": 0.05, "uso": "Análise de florações", "compartimento": "Compartimento B-01", "divisao_responsavel": "Divisão de Fiscalização e Segurança Química", "estoque_unidades": 40, "data_validade": "2027-01-20", "temperatura_c": -15, "estado_conservacao": "Estável", }, { "id_toxina": "TOX-0005", "nome": "Aflatoxina B1", "classe": "Micotoxina", "tipo_acao": "Carcinogênica", "periculosidade": 4, "dose_letal_mg": 65.0, "uso": "Monitoramento agroquímico", "compartimento": "Compartimento B-02", "divisao_responsavel": "Divisão de Toxicologia Ambiental", "estoque_unidades": 25, "data_validade": "2026-10-05", "temperatura_c": 2, "estado_conservacao": "Estável", }, { "id_toxina": "TOX-0006", "nome": "Toxina Botulínica Tipo A", "classe": "Proteína", "tipo_acao": "Neurotóxica", "periculosidade": 5, "dose_letal_mg": 0.001, "uso": "Produção de antitoxina", "compartimento": "Compartimento A-04", "divisao_responsavel": "Divisão de Pesquisa e Laboratório", "estoque_unidades": 3, "data_validade": "2027-06-01", "temperatura_c": -80, "estado_conservacao": "Crítico", }, { "id_toxina": "TOX-0007", "nome": "Alfa-amanitina", "classe": "Peptídeo", "tipo_acao": "Citotóxica", "periculosidade": 4, "dose_letal_mg": 0.15, "uso": "Estudos bioquímicos", "compartimento": "Compartimento B-03", "divisao_responsavel": "Divisão de Toxicologia Ambiental", "estoque_unidades": 18, "data_validade": "2026-12-18", "temperatura_c": -20, "estado_conservacao": "Estável", }, { "id_toxina": "TOX-0008", "nome": "Batracotoxina", "classe": "Alcaloide", "tipo_acao": "Cardiotóxica", "periculosidade": 5, "dose_letal_mg": 0.002, "uso": "Referência analítica", "compartimento": "Compartimento A-05", "divisao_responsavel": "Divisão de Emergências Químicas", "estoque_unidades": 2, "data_validade": "2027-02-14", "temperatura_c": -80, "estado_conservacao": "Crítico", }, { "id_toxina": "TOX-0009", "nome": "Brevetoxina", "classe": "Biotoxina marinha", "tipo_acao": "Neurotóxica", "periculosidade": 3, "dose_letal_mg": 1.0, "uso": "Controle de marés vermelhas", "compartimento": "Compartimento B-04", "divisao_responsavel": "Divisão de Fiscalização e Segurança Química", "estoque_unidades": 30, "data_validade": "2027-04-09", "temperatura_c": -10, "estado_conservacao": "Estável", }, { "id_toxina": "TOX-0010", "nome": "Solução padrão de cianeto", "classe": "Composto inorgânico", "tipo_acao": "Hemotóxica", "periculosidade": 2, "dose_letal_mg": 50.0, "uso": "Calibração de

equipamentos", "compartimento": "Compartimento C-01", "divisao_responsavel": "Serviços Gerais", "estoque_unidades": 60, "data_validade": "2028-01-30", "temperatura_c": 25, "estado_conservacao": "Estável", },] DIVISOES_MMA = ["Gabinete do Ministro", "Divisão de Toxicologia Ambiental", "Divisão de Emergências Químicas", "Divisão de Fiscalização e Segurança Química", "Divisão de Pesquisa e Laboratório", "Serviços Gerais",] def carregar_dados_toxinas(): """Retorna o inventário (simulado) de toxinas custodiadas no cofre do MMA.""" return pd.DataFrame(DADOS_TOXINAS) def serie_temporal_movimentacao(): """Gera uma série temporal mensal (simulada) de movimentações de toxinas.""" divisoes = ["Divisão de Toxicologia Ambiental", "Divisão de Emergências Químicas", "Divisão de Fiscalização e Segurança Química", "Divisão de Pesquisa e Laboratório",] meses = pd.date_range("2024-01-01", "2026-08-01", freq="MS") linhas = [] for i, mes in enumerate(meses): for idx, divisao in enumerate(divisoes): sazonalidade = (idx + 1) * math.sin(i / 6 * math.pi) tendencia = (i / len(meses)) * 4 mov = max(0, int((idx + 1) * 8 + sazonalidade + tendencia)) linhas.append({"divisao": divisao, "mes": mes, "movimentacoes": mov}) return pd.DataFrame(linhas) LIMITE_INATIVIDADE_S = 5 * 60 # 5 minutos sem interação => sessão expira def _sessao_valida(): """True se existe sessão autenticada e dentro do limite de inatividade.""" if not st.session_state.get("autenticado", False): return False usuario = st.session_state.get("usuario_logado") if usuario is None: return False login_ts = usuario.get("login_ts", 0) if time.time() - login_ts > LIMITE_INATIVIDADE_S: st.session_state.pop("usuario_logado", None) st.session_state.pop("autenticado", None) st.session_state["sessao_expirada"] = True return False return True def usuario_tem_nivel(nivel_requerido): """Verifica se o usuário logado possui o nível requerido (cumulativo).""" if not _sessao_valida(): return False usuario = st.session_state.get("usuario_logado") return usuario is not None and usuario.get("nivel_acesso", 0) >= nivel_requerido def usuario_pode_cadastrar(): """True apenas se o usuário autenticado tem permissão de cadastro.""" if not _sessao_valida(): return False usuario = st.session_state.get("usuario_logado") return usuario is not None and bool(usuario.get("pode_cadastrar", 0)) def existe_usuario_com_credenciais(): """True se ao menos um colaborador possui usuário cadastrado.""" try: conn = sqlite3.connect(DB_FILE) cur = conn.cursor() cur.execute("SELECT COUNT(*) FROM colaboradores " "WHERE usuario IS NOT NULL AND TRIM(usuario) <> '' ") n = cur.fetchone()[0] conn.close() return int(n) > 0 except Exception: return False def _preparar_logs(df_logs=None): """Carrega e normaliza os logs de acesso para o esquema atual do cofre.""" if df_logs is None: conn = sqlite3.connect(DB_FILE) df_logs = pd.read_sql_query("SELECT * FROM logs_acesso", conn) conn.close() df_logs =

```
df_logs.rename(columns={"setor": "area_dados"}) if df_logs.empty: return df_logs
df_logs["area_dados"] = df_logs["area_dados"].apply(normalizar_compartimento)
df_logs["nivel"] = ( df_logs["area_dados"].astype(str).str.extract(r"Nível\s*(\
d)").astype(int) ) df_logs["compartimento_curto"] =
df_logs["area_dados"].map( rotulo_curto_compartimento ) df_logs["data_hora"] =
pd.to_datetime(df_logs["data_hora"]) df_logs["data"] = df_logs["data_hora"].dt.date
df_logs["hora"] = df_logs["data_hora"].dt.hour return df_logs def _chart_png(chart,
largura=820, escala=2): """Renderiza um gráfico Altair para bytes PNG (em memória)."""
import vl_convert as vlc spec = chart.to_dict() spec["width"] = largura spec["autosize"] =
{"type": "fit", "contains": "padding"} return vlc.vegalite_to_png(spec, scale=escala) class
_RelatorioPDF(FPDF): def footer(self): self.set_y(-12) self.set_font(_PDF_FONT, "", 8)
self.set_text_color(148, 163, 184) self.cell(0, 8, f"Página {self.page_no()}", align="C")
_PDF_FONT = "dejavu" _DEJAVU_TTF = "/usr/share/fonts/truetype/dejavu/DejaVuSans.ttf"
_DEJAVU_BOLD_TTF = "/usr/share/fonts/truetype/dejavu/DejaVuSans-Bold.ttf" def
_registrar_fonte_pdf(pdf): """Registra a fonte Unicode (DejaVu) para suportar acentos e
travessões."""" if os.path.exists(_DEJAVU_TTF) and os.path.exists(_DEJAVU_BOLD_TTF):
pdf.add_font(_PDF_FONT, "", fname=_DEJAVU_TTF) pdf.add_font(_PDF_FONT, "B",
fname=_DEJAVU_BOLD_TTF) return True return False def _secao_pdf(pdf, numero, titulo):
if pdf.get_y() > 248: pdf.add_page() y = pdf.get_y() pdf.set_fill_color(30, 41, 59)
pdf.rect(pdf.l_margin, y, 180, 8, style="F") pdf.set_font(_PDF_FONT, "B", 11)
pdf.set_text_color(255, 255, 255) pdf.set_xy(pdf.l_margin + 3, y + 0.6) pdf.cell(180, 6,
f"{numero}. {titulo}") pdf.set_y(y + 10) def _inserir_kpi(pdf, x, y, w, h, titulo, valor):
pdf.set_fill_color(241, 245, 249) pdf.rect(x, y, w, h, style="F") pdf.set_text_color(71, 85, 105)
pdf.set_font(_PDF_FONT, "B", 7.5) pdf.set_xy(x + 3, y + 3) pdf.multi_cell(w - 6, 3.4,
titulo.upper()) pdf.set_font(_PDF_FONT, "B", 13) pdf.set_text_color(15, 118, 110)
pdf.set_xy(x + 3, y + h - 9.5) pdf.cell(w - 6, 6, str(valor)) pdf.set_y(y + h + 3) def
_row_kpis(pdf, kpis): n = len(kpis) gap = 3 w = (180 - gap * (n - 1)) / n h = 18 y = pdf.get_y()
if y + h + 8 > 262: pdf.add_page() y = pdf.get_y() for i, (titulo, valor) in enumerate(kpis):
_inserir_kpi(pdf, pdf.l_margin + i * (w + gap), y, w, h, titulo, valor) pdf.set_y(y + h + 4)
pdf.set_text_color(30, 41, 59) def _inserir_grafico(pdf, png, legenda): imagem =
Image.open(io.BytesIO(png)) w_px, h_px = imagem.size w = 180 h = w * h_px / w_px if
pdf.get_y() + h + 14 > 278: pdf.add_page() y = pdf.get_y() pdf.image(io.BytesIO(png),
x=pdf.l_margin, y=y, w=w, h=h) pdf.set_xy(pdf.l_margin, y + h + 1)
pdf.set_font(_PDF_FONT, "", 8.5) pdf.set_text_color(100, 116, 139) pdf.multi_cell(0, 4,
legenda) pdf.ln(2) def _tabela_pdf(pdf, titulo, headers, linhas, col_destaque=None,
mapa=None): pdf.set_font(_PDF_FONT, "B", 11) pdf.set_text_color(30, 41, 59) pdf.cell(0, 8,
```

titulo, new_x="LMARGIN", new_y="NEXT") pdf.ln(1) if not linhas: pdf.set_font(_PDF_FONT, "", 9.5) pdf.set_text_color(100, 116, 139) pdf.cell(0, 6, "Nenhum registro encontrado.", new_x="LMARGIN", new_y="NEXT") pdf.ln(3) return n = len(headers) lar = [180 / n] * n linha_h = 7 x0 = pdf.l_margin if pdf.get_y() + 12 > 270: pdf.add_page() pdf.set_fill_color(15, 23, 42) pdf.set_text_color(255, 255, 255) pdf.set_font(_PDF_FONT, "B", 8.5) x = x0 for i, h in enumerate(headers): pdf.set_xy(x, pdf.get_y()) pdf.cell(lar[i], linha_h, h, border=1, fill=True, align="C") x += lar[i] pdf.ln(linha_h) pdf.set_font(_PDF_FONT, "", 8.5) for j, linha in enumerate(linhas): if pdf.get_y() + 7 > 282: pdf.add_page() zebra = j % 2 == 1 x = x0 for i, val in enumerate(linha): fill = (248, 250, 252) if zebra else (255, 255, 255) txt = (33, 37, 41) if col_destaque is not None and i == col_destaque and mapa: m = mapa.get(val) if m: fill, txt = m pdf.set_fill_color(*fill) pdf.set_text_color(*txt) pdf.set_xy(x, pdf.get_y()) pdf.cell(lar[i], linha_h, str(val) if i == 0 else f" {val}", border=1, fill=True, align="L" if i == 0 else "C",) x += lar[i] pdf.ln(linha_h) pdf.ln(3) def gerar_relatorio_pdf(df_filtrado=None): """Gera um relatório PDF ilustrativo com KPIs, gráficos e tabelas. Se `df_filtrado` for informado, a seção de eventos de acesso reflete esses filtros; caso contrário usa todos os registros do banco. """ df_logs = _preparar_logs(df_filtrado) df_toxinas = carregar_dados_toxinas() df_serie = serie_temporal_movimentacao() conn = sqlite3.connect(DB_FILE) df_colabs = pd.read_sql_query("SELECT id, nome, cargo, divisao, nivel_acesso, data_cadastro " "FROM colaboradores", conn,) conn.close() total = len(df_logs) permitidos = int((df_logs["status"] == "PERMITIDO").sum()) negados = int((df_logs["status"] == "NEGADO").sum()) taxa = (permitidos / total * 100) if total else 0.0 dias = df_logs["data"].nunique() if total else 1 media_dia = total / dias if dias else 0.0 criticas = int((df_toxinas["estado_conservacao"] == "Crítico").sum()) agora = datetime.now() pdf = _RelatorioPDF(format="A4") pdf.set_margins(15, 15, 15) _registrar_fonte_pdf(pdf) pdf.add_page() # --- Cabeçalho --- pdf.set_fill_color(15, 23, 42) pdf.rect(0, 0, 210, 28, style="F") pdf.set_fill_color(13, 148, 136) pdf.rect(0, 28, 210, 2, style="F") pdf.set_text_color(255, 255, 255) pdf.set_font(_PDF_FONT, "B", 15) pdf.set_xy(10, 7) pdf.cell(0, 8, "COFRE DE TOXINAS — MINISTÉRIO DO MEIO AMBIENTE") pdf.set_font(_PDF_FONT, "", 9) pdf.set_xy(10, 16) pdf.cell(0, 5, "Sistema de Controle de Acesso Biométrico | Relatório de Segurança",) pdf.set_font(_PDF_FONT, "", 8) pdf.set_xy(10, 22) pdf.cell(0, 5, f"Gerado em {agora.strftime('%d/%m/%Y')} às {agora.strftime('%H:%M:%S')}",) pdf.set_y(34) if df_filtrado is not None: pdf.set_font(_PDF_FONT, "", 8.5) pdf.set_text_color(100, 116, 139) pdf.multi_cell(0, 4, "Obs.: este relatório reflete os filtros ativos no Dashboard Analítico.",) pdf.ln(2) pdf.set_text_color(30, 41, 59) # --- 1. Indicadores --- _secao_pdf(pdf, 1, "Indicadores Executivos") _row_kpis(pdf, [("Total de Acessos", total), ("Acessos Permitidos", permitidos), ("Tentativas Negadas", negados),],) _row_kpis(pdf, [("Taxa de Aprovação",

```
f"{taxa:.1f}%"), ("Média / dia", f"{media_dia:.1f}"), ("Compartimentos monitorados",
df_logs["nivel"].nunique() if total else 0), ], ) _row_kpis( pdf, [ ("Toxinas custodiadas",
len(df_toxinas)), ("Unidades em estoque", int(df_toxinas["estoque_unidades"].sum())),
("Estado crítico (toxinas)", criticas), ], ) # --- 2. Gráficos --- _secao_pdf(pdf, 2, "Dashboard
de Acessos — Gráficos Ilustrativos") pdf.set_font(_PDF_FONT, "B", 10)
pdf.set_text_color(30, 41, 59) pdf.cell(0, 6, "2.1 Tendência temporal de acessos",
new_x="LMARGIN", new_y="NEXT") pdf.ln(1) serie_dia = ( df_logs.groupby(["data",
"status"], as_index=False) .size() .rename(columns={"size": "contagem"}) ) serie_dia =
serie_dia.copy() serie_dia["data"] = serie_dia["data"].astype(str) chart_tendencia =
( alt.Chart(serie_dia) .mark_line(point=True, strokeWidth=2) .encode( x=alt.X("data:T",
title="Data"), y=alt.Y("contagem:Q", title="Nº de acessos"), color=alt.Color( "status:N",
title="Status", scale=alt.Scale( domain=["PERMITIDO", "NEGADO"], range=["#00C853",
"#D50000"], ), ), ) .properties(height=300, title="Acessos ao longo do tempo por status") )
_inserir_grafico( pdf, _chart_png(chart_tendencia), "Evolução diária dos eventos de
acesso, comparando acessos permitidos e negados.", ) pdf.set_font(_PDF_FONT, "B", 10)
pdf.set_text_color(30, 41, 59) pdf.cell( 0, 6, "2.2 Distribuição por compartimento do cofre",
new_x="LMARGIN", new_y="NEXT", ) pdf.ln(1) dist_area =
( df_logs.groupby(["compartimento_curto", "nivel"],
as_index=False) .size() .rename(columns={"size": "contagem"}) .sort_values(["nivel",
"compartimento_curto"]) ) chart_dist =
( alt.Chart(dist_area) .mark_bar() .encode( x=alt.X("contagem:Q", title="Nº de eventos"),
y=alt.Y("compartimento_curto:N", title="", sort="-x"), color=alt.Color( "nivel:O",
title="Nível", scale=alt.Scale( domain=["1", "2", "3"], range=["#2E7D32", "#F9A825",
"#C62828"], ), ), ) .properties(height=300, title="Eventos por nível de segurança") )
_inserir_grafico( pdf, _chart_png(chart_dist), ( "Eventos de acesso por nível (verde:
Permissão Geral, amarelo: " "Diretores de Divisões, vermelho: Ministro do Meio
Ambiente)." ), ) pdf.set_font(_PDF_FONT, "B", 10) pdf.set_text_color(30, 41, 59) pdf.cell(0, 6,
"2.3 Movimentação por hora do dia", new_x="LMARGIN", new_y="NEXT") pdf.ln(1)
serie_hora = ( df_logs.groupby(["hora", "status"],
as_index=False) .size() .rename(columns={"size": "contagem"}) ) chart_hora =
( alt.Chart(serie_hora) .mark_bar() .encode( x=alt.X("hora:O", title="Hora do dia"),
y=alt.Y("contagem:Q", title="Nº de eventos"), color=alt.Color( "status:N",
scale=alt.Scale( domain=["PERMITIDO", "NEGADO"], range=["#00C853",
"#D50000"], ), ), ) .properties(height=300, title="Acessos por horário") )
_inserir_grafico( pdf, _chart_png(chart_hora), "Volume de acessos por hora, útil para
identificar horários de pico e de risco.", ) pdf.set_font(_PDF_FONT, "B", 10)
```

```
pdf.set_text_color(30, 41, 59) pdf.cell( 0, 6, "2.4 Colaboradores mais ativos", new_x="LMARGIN", new_y="NEXT" ) pdf.ln(1) top_colabs = ( df_logs.groupby("colaborador_nome", as_index=False) .size() .rename(columns={"size": "contagem"}) .sort_values("contagem", ascending=False) .head(10) ) chart_top = ( alt.Chart(top_colabs) .mark_bar() .encode( x=alt.X("contagem:Q", title="Nº de eventos"), y=alt.Y("colaborador_nome:N", title="", sort="-x"), color=alt.Color("contagem:Q", scale=alt.Scale(scheme="viridis")), ) .properties(height=300, title="Top colaboradores") ) _inserir_grafico( pdf, _chart_png(chart_top), "Os dez colaboradores com o maior número de eventos de acesso registrados.", ) # --- 3. Panorama do cofre --- _secao_pdf(pdf, 3, "Panorama do Cofre de Toxinas") pdf.set_font(_PDF_FONT, "B", 10) pdf.set_text_color(30, 41, 59) pdf.cell( 0, 6, "3.1 Estoque de toxinas por classe química", new_x="LMARGIN", new_y="NEXT", ) pdf.ln(1) est_por_classe = ( df_toxinas.groupby("classe", as_index=False) .agg(estoque_unidades=("estoque_unidades", "sum")) ) chart_estoque = ( alt.Chart(est_por_classe) .mark_bar() .encode( x=alt.X("classe:N", title="", sort="-y"), y=alt.Y("estoque_unidades:Q", title="Unidades em estoque"), color=alt.Color("classe:N", legend=None), ) .properties(height=300, title="Unidades em estoque por classe") ) _inserir_grafico( pdf, _chart_png(chart_estoque), "Total de unidades custodiadas no cofre por classe de toxina.", ) pdf.set_font(_PDF_FONT, "B", 10) pdf.set_text_color(30, 41, 59) pdf.cell( 0, 6, "3.2 Série temporal de movimentações por divisão", new_x="LMARGIN", new_y="NEXT", ) pdf.ln(1) chart_serie = ( alt.Chart(df_serie) .mark_line(point=True) .encode( x=alt.X("mes:T", title="Mês / Ano"), y=alt.Y("movimentacoes:Q", title="Movimentações"), color=alt.Color("divisao:N", title="Divisão"), ) .properties(height=300, title="Movimentações mensais por divisão") ) _inserir_grafico( pdf, _chart_png(chart_serie), "Série mensal simulada de movimentações de toxinas por divisão do ministério.", ) # --- 4. Logs de Acesso --- _secao_pdf(pdf, 4, "Logs de Acesso") mapa_status = { "PERMITIDO": ((220, 252, 231), (22, 101, 52)), "NEGADO": ((254, 226, 226), (153, 27, 27)), } logs_linhas = [ (r["data_hora"].strftime("%d/%m/%Y %H:%M"), r["colaborador_nome"], r["compartimento_curto"], r["nivel"], r["status"]) for _, r in df_logs.sort_values("data_hora", ascending=False).iterrows() ] _tabela_pdf( pdf, "Eventos de acesso registrados", ["Data / Hora", "Colaborador", "Compartimento", "Nível", "Status"], logs_linhas, col_destaque=4, mapa=mapa_status, ) # --- 5. Colaboradores --- _secao_pdf(pdf, 5, "Equipe do Cofre") colabs_linhas = [ (r["nome"], r["cargo"] or "—", r["divisao"] or "Sem divisão", rotulo_nivel(r["nivel_acesso"])) for _, r in df_colabs.iterrows() ] _tabela_pdf( pdf, "Colaboradores cadastrados", ["Nome", "Cargo / Função", "Divisão", "Nível de Acesso"], colabs_linhas, ) # --- 6. Toxinas --- _secao_pdf(pdf, 6, "Catálogo de Toxinas") tox_linhas = [ (r["id_toxina"], r["nome"], r["classe"],
```

r["compartimento"], r["estoque_unidades"], r["estado_conservacao"]) for _, r in df_toxinas.iterrows()] _tabela_pdf(pdf, "Substâncias custodiadas no cofre", ["ID", "Toxina", "Classe", "Compartimento", "Estoque", "Estado"], tox_linhas,) # --- 7. Cadeia de Custódia --- _secao_pdf(pdf, 7, "Cadeia de Custódia") custodia = pd.DataFrame([{"lote": "TOX-0006/2026", "toxina": "Toxina Botulínica Tipo A", "de": "Compartimento A-04", "para": "Laboratório Central", "responsavel": "Diretor de Pesquisa", "data_hora": "2026-08-14 09:32"}, {"lote": "TOX-0001/2026", "toxina": "Ricina", "de": "Compartimento A-01", "para": "Sala de Análises", "responsavel": "Diretor de Toxicologia", "data_hora": "2026-08-15 14:05"}, {"lote": "TOX-0004/2026", "toxina": "Microcistina-LR", "de": "Compartimento B-01", "para": "Incinerador controlado", "responsavel": "Ministro do Meio Ambiente", "data_hora": "2026-09-02 10:12"},]) custodia_linhas = [(r["lote"], r["toxina"], r["de"], r["para"], r["responsavel"], r["data_hora"]) for _, r in custodia.iterrows()] _tabela_pdf(pdf, "Movimentações de substâncias (exemplo)", ["Lote", "Toxina", "De", "Para", "Responsável", "Data / Hora"], custodia_linhas,) return bytes(pdf.output()) def recortar_centro_rosto(img_path): """Corta a área central focando no rosto.""" try: with Image.open(img_path) as img: w, h = img.size left = int(w * 0.20) top = int(h * 0.05) right = int(w * 0.80) bottom = int(h * 0.85) crop_img = img.crop((left, top, right, bottom)) crop_img.save(img_path) except Exception as e: registrar_erro("IMAGEM", "Processamento de Imagem", str(e)) _DETECTOR_MTCNN = None def _detector_mtcnn(): global _DETECTOR_MTCNN if _DETECTOR_MTCNN is None: from mtcnn import MTCNN _DETECTOR_MTCNN = MTCNN() return _DETECTOR_MTCNN def _imgs_bytes_para_numpy(frames_bytes): """Converte uma lista de bytes (JPEG) em imagens RGB (numpy).""" imgs = [] for b in frames_bytes: arr = cv2.imdecode(np.frombuffer(b, np.uint8), cv2.IMREAD_COLOR) if arr is not None: imgs.append(cv2.cvtColor(arr, cv2.COLOR_BGR2RGB)) return imgs def _regiao_uniao(a, b): ax, ay, aw, ah = a bx, by, bw, bh = b x1 = min(ax, bx) y1 = min(ay, by) x2 = max(ax + aw, bx + bw) y2 = max(ay + ah, by + bh) return x1, y1, x2, y2 # ---------- Liveness V2 — anti-spoofing reforçado ---------- LIMIAR_MOVIMENTO = 0.015 LIMIAR_N_COMPOS = 4 LIMIAR_PARALAXE = 0.05 LIMIAR_PISCADA = 0.60 # abertura do olho cai abaixo de 60% do baseline = piscada JANELA_MINIMA_S = 2.0 LIMIAR_TELA_FFT = 12.0 def _regiao_uniao(a, b): ax, ay, aw, ah = a bx, by, bw, bh = b x1 = min(ax, bx) y1 = min(ay, by) x2 = max(ax + aw, bx + bw) y2 = max(ay + ah, by + bh) return x1, y1, x2, y2 _DLIB_DETECTOR = None _DLIB_PREDICTOR = None def _carregar_preditor_dlib(): """Carrega o detector/predictor de 68 landmarks do dlib (uma vez).""" global _DLIB_DETECTOR, _DLIB_PREDICTOR if _DLIB_PREDICTOR is not None: return _DLIB_PREDICTOR import dlib from site import getsitepackages, getusersitepackages

candidato = None for raiz in getsitepackages() + [getusersitepackages()]: achado = glob.glob(os.path.join(raiz, "**", "shape_predictor_68_face_landmarks.dat"), recursive=True,) if achado: candidato = achado[0] break if not candidato: raise FileNotFoundError("shape_predictor_68_face_landmarks.dat não encontrado.") _DLIB_DETECTOR = dlib.get_frontal_face_detector() _DLIB_PREDICTOR = dlib.shape_predictor(candidato) return _DLIB_PREDICTOR def _ear_dos_olhos(cinza): """EAR (Eye Aspect Ratio) por olho via dlib 68 landmarks. Olho aberto ~0.25-0.33; piscado <-0.12. Retorna (esq, dir) ou None.""" try: pred = _carregar_preditor_dlib() except Exception as e: registrar_erro("BIOMETRIA", "Dlib Landmarks", str(e)) return None faces = _DLIB_DETECTOR(cinza) if not faces: return None face = max(faces, key=lambda r: r.width() * r.height()) try: shape = pred(cinza, face) except Exception: return None def ear(indices): pts = [np.array((shape.part(i).x, shape.part(i).y)) for i in indices] return (np.linalg.norm(pts[1] - pts[5]) + np.linalg.norm(pts[2] - pts[4])) / (2.0 * np.linalg.norm(pts[0] - pts[3]) + 1e-6) return ear([36, 37, 38, 39, 40, 41]), ear([42, 43, 44, 45, 46, 47]) def _piscada_detectada(ears_por_quadro): """True se algum olho piscou (EAR caiu abaixo de 60% do baseline aberto). None se não houve landmarks suficientes (inconclusivo)""" validos = [e for e in ears_por_quadro if e is not None] if not validos: return None esq = [e[0] for e in validos] dir_ = [e[1] for e in validos] for serie in (esq, dir_): if len(serie) < 2: continue baseline = max(serie) if baseline < 0.20: continue if min(serie) < LIMIAR_PISCADA * baseline: return True return False def _variacao_paralaxe(metas): """Variação das proporções geométricas (nariz↔olhos↔boca) ao longo dos quadros. Rosto real girando muda essas proporções; foto plana deslizando mantém as proporções constantes (pura translação 2D).""" of_x, dy_boca = [], [] for meta in metas: kp = meta["kp"] le = np.array(kp["left_eye"], dtype=np.float64) re = np.array(kp["right_eye"], dtype=np.float64) nose = np.array(kp["nose"], dtype=np.float64) ml = np.array(kp["mouth_left"], dtype=np.float64) mr = np.array(kp["mouth_right"], dtype=np.float64) eye_w = float(np.linalg.norm(re - le)) if eye_w < 10: continue centro_x = 0.5 * (le[0] + re[0]) centro_y = 0.5 * (le[1] + re[1]) of_x.append((nose[0] - centro_x) / eye_w) dy_boca.append((0.5 * (ml[1] + mr[1]) - centro_y) / eye_w) if not of_x: return 0.0 return float(np.std(of_x) + np.std(dy_boca)) def _detectar_tela(cinza, face_box): """Força de padrão periódico de alta frequência (Moiré/grade de tela) no recorte do rosto via FFT. Valor alto indica imagem vinda de uma tela.""" bx, by, bw, bh = face_box h, w = cinza.shape[:2] bx1, by1 = max(bx, 0), max(by, 0) bx2, by2 = min(bx + bw, w), min(by + bh, h) crop = cinza[by1:by2, bx1:bx2] if crop.size < 32 * 32: return 0.0 g = cv2.resize(crop, (256, 256)).astype(np.float32) janela_hann = np.outer(np.hanning(256), np.hanning(256)).astype(np.float32) g = g * janela_hann mag =

np.abs(np.fft.fftshift(np.fft.fft2(g))) H, W = mag.shape yy, xx = np.mgrid[0:H, 0:W] d = np.sqrt((yy - H // 2) ** 2 + (xx - W // 2) ** 2) raio_max = 0.47 * H anulo = (d > 0.55 * raio_max) & (d <= raio_max) if anulo.sum() < 64: return 0.0 an = mag[anulo] media = float(an.mean()) if media <= 1e-9: return 0.0 return float(an.max() / media) def avaliar_liveness(frames): """V2: 6 barreiras anti-fraude. `frames` = [(bytes_do_quadro, timestamp)]. Retorna (aprovado, motivo, detalhes, idx_melhor_quadro). Barreiras: janela >= 2s; movimento; padrão orgânico temporal; variação geométrica (parallax 3D); >= 1 piscada; ausência de tela (Moiré). """ imgs, ts = [], [] for b, t in frames: arr = cv2.imdecode(np.frombuffer(b, np.uint8), cv2.IMREAD_COLOR) if arr is not None: imgs.append(cv2.cvtColor(arr, cv2.COLOR_BGR2RGB)) ts.append(t) detalhes = {"quadros_capturados": len(imgs)} if len(imgs) < 3: return False, "É necessário capturar ao menos 3 quadros.", detalhes, 0 if len(ts) >= 2 and (ts[-1] - ts[0]) < JANELA_MINIMA_S: return (False, "Captura rápida demais (menos de 2s entre o 1º e o último quadro) " "— típico de rajada de fotos, sem interação real.", detalhes, 0,) detector = _detector_mtcnn() grays, boxes, metas = [], [], [] for arr in imgs: hgt, wdt = arr.shape[:2] escala = 360.0 / float(hgt) pequeno = cv2.resize(arr, (int(wdt * escala), 360)) cinza = cv2.cvtColor(pequeno, cv2.COLOR_RGB2GRAY) faces = detector.detect_faces(pequeno) meta = None if faces: face = max(faces, key=lambda f: f["box"][2] * f["box"][3]) x, y, w, h2 = face["box"] if w >= 30 and h2 >= 30: meta = {"box": tuple(face["box"]), "kp": face["keypoints"]} grays.append(cinza) metas.append(meta) boxes.append(meta["box"] if meta else None) presentes = [i for i, m in enumerate(metas) if m is not None] n_req = max(2, int(round(len(imgs) * 0.6))) detalhes["rostos_detectados"] = len(presentes) if len(presentes) < n_req: return (False, "Rosto não detectado em quadros suficientes (imagem borrada, " "escura ou fora do círculo).", detalhes, 0,) diffs = [] for k in range(len(presentes) - 1): a, b = presentes[k], presentes[k + 1] x1, y1, x2, y2 = _regiao_uniao(boxes[a], boxes[b]) mov = (float(np.mean(cv2.absdiff(grays[a], grays[b])[y1:y2, x1:x2])) / 255.0) diffs.append(mov) movimento_max = max(diffs) detalhes["movimento_máximo"] = f"{movimento_max * 100:.2f}%" if movimento_max < LIMIAR_MOVIMENTO: return (False, "**Movimento insuficiente do rosto** entre os quadros — típico de " "foto ou tela parada exibida à câmera. Repita o mini vídeo movendo " "a cabeça e piscando os olhos a cada captura.", detalhes, 0,) regioes = [] for idx in presentes: bx, by, bw, bh = boxes[idx] crop = grays[idx] [by : by + bh, bx : bx + bw] regioes.append(cv2.resize(crop, (80, 80))) pilha = np.stack(regioes).astype(np.float32) var_map = pilha.var(axis=0) q75 = np.percentile(var_map, 75) alto = (var_map >= q75).astype(np.uint8) if q75 > 0 else None n_comps = 0 if alto is not None: n_comps = cv2.connectedComponents(alto, connectivity=8)[0] detalhes["regiões_de_variação"] = int(n_comps) if n_comps <

LIMIAR_N_COMPOS: detalhes["padrão_de_movimento"] = "Rígido / uniforme (possível foto)" return (False, "**Movimento rígido e plano** — a imagem se moveu como uma única " "peça (foto ou tela deslizando), sem o padrão orgânico de um rosto " "real. Repita girando a cabeça de verdade (olhe para os lados) e " "pisque.", detalhes, 0,) paralaxe = (_variacao_paralaxe([metas[i] for i in presentes]) if presentes else 0.0) detalhes["variação_geométrica_(parallax)"] = f"{paralaxe:.3f}" if paralaxe < LIMIAR_PARALAXE: detalhes["padrão_de_movimento"] = "Rígido / uniforme (possível foto)" return (False, "**Sem variação 3D da face** — as proporções entre nariz/olhos/" "boca não mudaram durante o movimento (translação rígida, típica " "de foto deslizando). Gire a cabeça para os lados.", detalhes, 0,) ears_por_quadro = [] for idx in presentes: ears_por_quadro.append(_ear_dos_olhos(grays[idx])) piscou = _piscada_detectada(ears_por_quadro) detalhes["piscada_detectada"] = { True: "Sim", False: "Não", None: "Inconclusivo", }[piscou] if piscou is None: return (False, "**Não foi possível analisar os olhos** — aproxime o rosto do " "círculo, olhe de frente e em boa iluminação para repetir.", detalhes, 0,) if not piscou: return (False, "**Nenhuma piscada detectada** — é necessário **piscar** (fechar e " "abrir os olhos) durante as capturas. Fotos e telas nunca piscam.", detalhes, 0,) melhor = max(presentes, key=lambda i: (boxes[i][2] * boxes[i][3])) sinal_tela = _detectar_tela(grays[melhor], boxes[melhor]) detalhes["sinal_de_tela_(moiré)"] = f"{sinal_tela:.1f}" if sinal_tela >= LIMIAR_TELA_FFT: return (False, "**Padrão de tela detectado (Moiré)** — a imagem parece ter sido " "exibida em um monitor/celular diante da câmera. Aproxime-se " "iluminando o rosto diretamente e evite telas.", detalhes, melhor,) detalhes["padrão_de_movimento"] = "Orgânico (faces reais)" return True, "Movimento 3D do rosto + piscada detectados.", detalhes, melhor def _melhor_match_biometrico(temp_test_path): """Busca o colaborador cadastrado mais próximo (VGG-Face / cosine)."""" conn = sqlite3.connect(DB_FILE) cur = conn.cursor() cur.execute("SELECT id, nome, foto_path FROM colaboradores") cadastrados = cur.fetchall() conn.close() melhor_candidato = None menor_distancia = 1.0 for cid, cnome, cpath in cadastrados: if not os.path.exists(cpath): continue try: res = DeepFace.verify(img1_path=temp_test_path, img2_path=cpath, model_name="VGG-Face", detector_backend="skip", distance_metric="cosine", enforce_detection=False,) distancia = res.get("distance", 1.0) if distancia < menor_distancia: menor_distancia = distancia melhor_candidato = (cid, cnome) except Exception as e: registrar_erro("BIOMETRIA", f"Validação Biométrica ({cnome})", str(e)) return melhor_candidato, menor_distancia def _conceder_ou_negar(colab_id, colab_nome, menor_distancia, nivel_requerido, area_requerida): """Decisão final pós-reconhecimento: nível, login e logs."""" estava_autenticado =

st.session_state.get("autenticado", False) conn = sqlite3.connect(DB_FILE) cur = conn.cursor() cur.execute("SELECT nivel_acesso, usuario, pode_cadastrar " "FROM colaboradores WHERE id = ?", (colab_id,),) linha_nivel = cur.fetchone() conn.close() nivel_colab = int(linha_nivel[0]) if linha_nivel else 1 usuario_credencial = linha_nivel[1] if linha_nivel else None pode_cadastrar = int(linha_nivel[2] or 0) if linha_nivel else 0 if nivel_colab >= nivel_requerido: st.success(f"✅ **ACESSO AUTORIZADO!**\n\nBem-vindo(a), **{colab_nome}**! " f"(Distância: {menor_distancia:.3f})\nCompartimento liberado: " f"**{area_requerida}**") registrar_log(colab_nome, area_requerida, "PERMITIDO") st.session_state.pop("sessao_expirada", None) st.session_state["usuario_logado"] = { "id": colab_id, "nome": colab_nome, "nivel_acesso": nivel_colab, "usuario": usuario_credencial, "pode_cadastrar": pode_cadastrar, "login_ts": time.time(), "metodo": "biometria", } st.session_state["autenticado"] = True st.toast("🔓 Sessão iniciada. Compartimentos liberados conforme o nível do colaborador.", icon="🔓",) if not estava_autenticado: st.session_state["redirecionar_para"] = "🔐 Cofre de Toxinas do MMA" st.rerun() if not usuario_credencial: st.info("i **Acesso por biometria.** Este colaborador ainda não possui " "usuário/senha definidos. Solicite ao nível 3 (Ministro) que " "defina as credenciais na aba 🛡️ **Gestão de Acessos**.") return True st.error(f"🚫 **ACESSO NEGADO!**\n\nColaborador identificado (**{colab_nome}**), " f"porém com nível `{nivel_colab}` (não possui o nível `{nivel_requerido}` " f"exigido para o compartimento `{area_requerida}`).") registrar_log(colab_nome, area_requerida, "NEGADO", {"motivo": "Nível insuficiente", "nivel_colab": nivel_colab},) _registrar_e_verificar_bloqueio(area_requerida, f"Acesso indevido: {colab_nome} (nível {nivel_colab}) tentou o " f"compartimento de nível {nivel_requerido}.", origem="foto",) return False # ---------- Política de bloqulo temporário por compartimento ---------- JANELA_FRAUDE_S = 10 * 60 # janela de contagem de tentativas (10 min) DURACAO_BLOQUEIO_S = 15 * 60 # bloqueio após limite (15 min) MAX_TENTATIVAS = 3 # tentativas de fraude/negadas p/ disparar bloqueio def _registrar_tentativa_fraude(compartimento, motivo, detalhes=None, origem="mini_video"): """Registra uma tentativa de fraude/negada e retorna o nº na janela.""" conn = sqlite3.connect(DB_FILE) cur = conn.cursor() dh = datetime.now().strftime("%Y-%m-%d %H:%M:%S") cur.execute("INSERT INTO tentativas_seguranca " "(compartimento, data_hora, motivo, detalhes, origem) " "VALUES (?, ?, ?, ?, ?)", (compartimento, dh, motivo, json.dumps(detalhes or {}, ensure_ascii=False), origem,),) conn.commit() desde = (datetime.now() - timedelta(seconds=JANELA_FRAUDE_S)).strftime("%Y-%m-%d %H:%M:%S") cur.execute("SELECT COUNT(*) FROM tentativas_seguranca " "WHERE compartimento = ?

AND data_hora >= ?", (compartimento, desde),) n = cur.fetchone()[0] conn.close() return int(n) def _bloqueio_ativo(compartimento): """Retorna (bloqueado, minutos_restantes) para o compartimento."""" conn = sqlite3.connect(DB_FILE) cur = conn.cursor() desde = (datetime.now() - timedelta(seconds=JANELA_FRAUDE_S)).strftime("%Y-%m-%d %H:%M: %S") cur.execute("SELECT COUNT(*) FROM tentativas_seguranca " "WHERE compartimento = ? AND data_hora >= ?", (compartimento, desde),) n = cur.fetchone()[0] encerra_em = None if n >= MAX_TENTATIVAS: cur.execute("SELECT data_hora FROM tentativas_seguranca " "WHERE compartimento = ? AND data_hora >= ? ORDER BY id DESC LIMIT 1", (compartimento, desde),) ultima = cur.fetchone() if ultima: t_ultima = datetime.strptime(ultima[0], "%Y-%m-%d %H:%M:%S") encerra_em = t_ultima + timedelta(seconds=DURACAO_BLOQUEIO_S) conn.close() if encerra_em is None: return False, 0 restantes = (encerra_em - datetime.now()).total_seconds() if restantes <= 0: return False, 0 minutos = int(restantes // 60) + (1 if restantes % 60 else 0) return True, minutos def _tentativas_recentes(compartimento, limite=8): """Lista das últimas tentativas de fraude/negadas do compartimento."""" conn = sqlite3.connect(DB_FILE) cur = conn.cursor() cur.execute("SELECT data_hora, motivo, origem FROM tentativas_seguranca " "WHERE compartimento = ? ORDER BY id DESC LIMIT ?", (compartimento, limite),) linhas = [{"Data/Hora": dh, "Motivo": m, "Origem": o} for dh, m, o in cur.fetchall()] conn.close() return linhas def _registrar_e_verificar_bloqueio(compartimento, motivo, detalhes=None, origem="mini_video"): """Registra a tentativa; se atingiu o limite, aciona o bloqueio."""" n = _registrar_tentativa_fraude(compartimento, motivo, detalhes, origem) bloq, minutos = _bloqueio_ativo(compartimento) if bloq and st.session_state.get("alertou_bloqueio") != compartimento: st.session_state["alertou_bloqueio"] = compartimento return bloq, minutos def _render_mini_video(nivel_requerido, area_requerida): """UI do mini vídeo anti-fraude (liveness) + validação biométrica."""" st.info("🎥 **Mini vídeo anti-fraude.** Posicione o rosto no círculo e capture " "**de 4 a 8 quadros**, **girando a cabeça** para os lados/cima-baixo e " "**piscando** durante as capturas (as fotos nunca piscam). O sistema " "exige >2s de captura, movimento orgânico 3D, ao menos uma piscada e " "ausência de tela (Moiré) — **fotos na tela do celular serão bloqueadas**.") if "live_frames" not in st.session_state: st.session_state["live_frames"] = [] frame = st.camera_input("🎥 Capture os quadros do mini vídeo (mova o rosto entre as capturas)", key="live_cam_webcam",) if frame is not None: novo = frame.getvalue() fichas = st.session_state["live_frames"] if len(fichas) < 8 and (not fichas or hashlib.sha1(novo).hexdigest() != hashlib.sha1(fichas[-1][0]).hexdigest()): fichas.append((novo, time.time())) st.toast(f"📸 Quadro {len(fichas)} capturado — mova a

cabeça e capture mais!", icon="🎥",) n = len(st.session_state["live_frames"]) c1, c2 = st.columns([1, 1]) c1.metric("Quadros capturados", f"{n}/8") if c2.button("🧹 Limpar quadros", use_container_width=True): st.session_state["live_frames"] = [] st.session_state["alertou_bloqueio"] = None st.rerun() if st.session_state["live_frames"]: thumbs = [Image.open(io.BytesIO(b)).copy() for b, _ in st.session_state["live_frames"]] thumbs = [t.resize((90, int(90 * t.height / t.width))) for t in thumbs] st.image(thumbs, caption=[f"Q{i+1}" for i in range(len(thumbs))], width=90,) if st.button("🔍 Analisar Mini Vídeo e Validar Acesso", type="primary", disabled=(n < 3),): aprovado, motivo, detalhes, idx_melhor = avaliar_liveness(st.session_state["live_frames"]) if not aprovado: st.error(f"🚨 **ACESSO NEGADO — FRAUDE SUSPEITA!**\n\n{motivo}\n\n" f"Detalhes: {detalhes}") registrar_log("Desconhecido", area_requerida, "NEGADO", {"motivo": motivo, **detalhes},) bloq, minutos = _registrar_e_verificar_bloqueio(area_requerida, f"Mini vídeo bloqueado: {motivo}", detalhes, origem="mini_video",) if bloq: st.error(f"⛔ **BLOQUEIO TEMPORÁRIO ATIVADO** neste compartimento. " f"Nova validação permitida em até {minutos} min.") else: return st.success(f"✅ **Liveness aprovado!** {motivo}\n\nDetalhes: {detalhes}") temp_test_path = "temp_test.jpg" with open(temp_test_path, "wb") as f: f.write(st.session_state["live_frames"][idx_melhor][0]) recortar_centro_rosto(temp_test_path) try: carregar_modelo_facial() with st.spinner("Validando biometria facial no melhor quadro..."): melhor_candidato, menor_distancia = _melhor_match_biometrico(temp_test_path) if os.path.exists(temp_test_path): os.remove(temp_test_path) if melhor_candidato is not None and menor_distancia <= 0.35: colab_id, colab_nome = melhor_candidato _conceder_ou_negar(colab_id, colab_nome, menor_distancia, nivel_requerido, area_requerida,) else: st.error(f"🚨 **ACESSO NEGADO!** Biometria não bateu com nenhum " f"cadastrado. (Menor distância: {menor_distancia:.3f})") registrar_log("Desconhecido", area_requerida, "NEGADO", {"motivo": "Biometria não reconhecida (mini vídeo)"},) _registrar_e_verificar_bloqueio(area_requerida, "Biometria não reconhecida (mini vídeo).", origem="mini_video",) except Exception as e: registrar_erro("GERAL", "Validação de Acesso (Mini Vídeo)", str(e)) st.error(f"❌ Ocorreu um erro inesperado na validação. Veja a aba 🐞 " f"Registro de Erros para ajuda. ({e})") def registrar_log(nome, area, status, detalhes=None): conn = sqlite3.connect(DB_FILE) cursor = conn.cursor() dh = datetime.now().strftime("%Y-%m-%d %H:%M:%S") cursor.execute("INSERT INTO logs_acesso " "(colaborador_nome, setor, data_hora, status, detalhes) " "VALUES (?, ?, ?, ?, ?)", (nome, area, dh, status, json.dumps(detalhes, ensure_ascii=False) if detalhes else None,),) conn.commit() conn.close() def obter_ajuda_erro(tipo, mensagem): """Retorna (descricao, sugerida) amigáveis para o

tipo/mensagem do erro."""
m = (mensagem or "").lower()
generica = (
"Ocorreu um erro inesperado durante a operação.",
"Verifique o texto técnico abaixo e, se persistir, revise a "
"configuração do sistema ou contate o suporte técnico.",
)
ajuda = {
"IMAGEM": [
(("cannot identify", "not an image", "cannot open", "unidentified image"),
"O arquivo enviado não é uma imagem válida ou está corrompido "
"(uso do formato JPG/PNG esperado).",
"Utilize uma imagem JPG ou PNG válida e nítida.",
),
],
"BIOMETRIA": [
(("verify", "no face", "face", "detect", "enforce_detection"),
"Não foi possível processar a identificação facial da imagem "
"(rosto pouco nítido ou ausente).",
"Centralize o rosto na câmera com boa iluminação e tente "
"novamente.",
),
],
"BANCO": [
(("no such table", "database is locked", "operationalerror", "unable to open database"),
"Problema ao acessar o banco de dados local "
"(controle_acesso.db).",
"Verifique se o arquivo existe e não está bloqueado por outro "
"processo. Reinicie a aplicação se necessário.",
),
],
}
for tipo_chave, regras in ajuda.items():
if tipo != tipo_chave:
continue
for (padroes, desc, sug) in regras:
if any(p in m for p in padroes):
return desc, sug
return generica
def registrar_erro(tipo, modulo, mensagem):
"""Registra um erro na tabela logs_erros com ajuda amigável associada."""
try:
descricao, sugerida = obter_ajuda_erro(tipo, mensagem)
dh = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
conn = sqlite3.connect(DB_FILE)
cursor = conn.cursor()
cursor.execute(
"INSERT INTO logs_erros (data_hora, tipo, modulo, mensagem, "
"descricao, sugerida) VALUES (?, ?, ?, ?, ?, ?)",
(dh, tipo, modulo, mensagem, descricao, sugerida),
)
conn.commit()
conn.close()
except Exception:
pass
def limpar_logs_erros():
try:
conn = sqlite3.connect(DB_FILE)
cursor = conn.cursor()
cursor.execute("DELETE FROM logs_erros")
conn.commit()
conn.close()
except Exception:
pass
def salvar_colaborador():
nome = st.session_state.get("reg_nome", "").strip()
cargo = st.session_state.get("reg_cargo", "").strip()
divisao = st.session_state.get("reg_divisao", "Sem divisão")
nivel = st.session_state.get("reg_nivel", 1)
usuario = st.session_state.get("reg_usuario", "").strip()
senha = st.session_state.get("reg_senha", "")
foto_bytes = st.session_state.get("foto_capturada_bytes", None)
if not nome or foto_bytes is None:
st.session_state["msg_status"] = (
"error",
"⚠️ Preencha o Nome e tire/envie a Foto antes de cadastrar.",
)
return
if not usuario:
st.session_state["msg_status"] = (
"error",
"⚠️ Defina o Usuário (login) do colaborador.",
)
return
if len(senha) < 6:
st.session_state["msg_status"] = (
"error",
"⚠️ A senha deve ter ao menos 6 caracteres.",
)
return
try:
conn = sqlite3.connect(DB_FILE)
cursor = conn.cursor()
cursor.execute(
"SELECT COUNT(*) FROM colaboradores WHERE usuario = ? COLLATE NOCASE",
(usuario,),
)
if cursor.fetchone()[0] > 0:
conn.close()
st.session_state["msg_status"] = (
"error",
"⚠️ Este usuário já está em uso por outro

colaborador.",) return filename = (f"{nome.replace(' ', '_').lower()}_{int(datetime.now().timestamp())}.jpg") foto_path = os.path.join(FOTOS_DIR, filename) with open(foto_path, "wb") as f: f.write(foto_bytes) recortar_centro_rosto(foto_path) senha_hash = hash_senha(senha) data_hoje = datetime.now().strftime("%Y-%m-%d") cursor.execute("INSERT INTO colaboradores " "(nome, cargo, divisao, nivel_acesso, usuario, senha_hash, pode_cadastrar, " "foto_path, data_cadastro) " "VALUES (?, ?, ?, ?, ?, ?, 0, ?, ?)", (nome, cargo, divisao, nivel, usuario, senha_hash, foto_path, data_hoje),) conn.commit() conn.close() st.session_state["reg_nome"] = "" st.session_state["reg_cargo"] = "" st.session_state["reg_divisao"] = "Sem divisão" st.session_state["reg_nivel"] = 1 st.session_state["reg_usuario"] = "" st.session_state["reg_senha"] = "" st.session_state.pop("foto_capturada_bytes", None) st.session_state["msg_status"] = ("success", f"✅ Colaborador **{nome}** cadastrado com sucesso! " f"Nível de acesso: **{rotulo_nivel(nivel)}**. Credenciais criadas " f"para o usuário **{usuario}**.",) except Exception as e: registrar_erro("BANCO", "Cadastro de Colaborador", str(e)) st.session_state["msg_status"] = ("error", f"❌ Erro ao cadastrar colaborador. Veja a aba 🐞 Registro de Erros para ajuda. ({e})",) @st.cache_resource(show_spinner=False) def carregar_modelo_facial(): """Carrega o modelo VGG-Face uma única vez (evita recarregar a cada validação)""" try: return DeepFace.build_model("VGG-Face") except Exception as e: registrar_erro("BIOMETRIA", "Carregamento do Modelo Facial", str(e)) return None # -- # AUTENTICAÇÃO POR CREDENCIAIS (usuário/senha) # -- def _autenticar(dados): """Valida as credenciais no banco e inicia a sessão. Retorna (ok, msg).""" usuario = dados.get("login").strip() senha = dados.get("senha") if not usuario or not senha: return False, "Informe usuário e senha." try: conn = sqlite3.connect(DB_FILE) cur = conn.cursor() cur.execute("SELECT id, nome, cargo, divisao, nivel_acesso, usuario, pode_cadastrar, senha_hash " "FROM colaboradores " "WHERE usuario = ? COLLATE NOCASE", (usuario,),) linha = cur.fetchone() conn.close() except Exception as e: registrar_erro("AUTENTICACAO", "Login", str(e)) return False, "Erro interno ao validar o login." if linha is None or not verificar_senha(senha, linha[7]): registrar_log("Desconhecido", "Sistema – Login", "NEGADO", {"motivo": "Credenciais inválidas", "usuario_tentado": usuario},) return False, "Usuário ou senha inválidos." _id, _nome, _cargo, _divisao, _nivel, _usuario, _pode_cad, _ = linha st.session_state.pop("sessao_expirada", None) st.session_state["usuario_logado"] = { "id": _id, "nome": _nome, "cargo": _cargo, "divisao": _divisao, "nivel_acesso": int(_nivel), "usuario": _usuario, "pode_cadastrar": int(_pode_cad or 0), "login_ts": time.time(),

"metodo": "credenciais", } st.session_state["autenticado"] = True registrar_log(f"{_nome} (login)", rotulo_nivel(_nivel), "PERMITIDO", {"motivo": "Login por credenciais"},) return True, f"Bem-vindo(a), {_nome}!" def _encontrar_imagem_home(): """Retorna o caminho da primeira imagem encontrada em IMAGEMHOME/ (ou None).""" if not os.path.isdir(IMAGEMHOME_DIR): return None for f in os.listdir(IMAGEMHOME_DIR): if f.lower().endswith(EXTENSOES_IMAGEM): return os.path.join(IMAGEMHOME_DIR, f) return None @st.cache_data def _imagem_home_base64(_caminho): """Codifica a imagem da home em base64 (cacheado).""" with open(_caminho, "rb") as f: return base64.b64encode(f.read()).decode("ascii") def _zonas_home_default(): """Zonas clicáveis padrão (percentuais sobre a imagem da home).""" return [{"chave": "campo", "rotulo": "Usuário e Senha", "acao": "login", "x": 62.0, "y": 40.0, "w": 22.0, "h": 17.0}, {"chave": "acesso", "rotulo": "Acesso", "acao": "acesso", "x": 61.0, "y": 57.0, "w": 23.0, "h": 10.0}, {"chave": "cadastro", "rotulo": "Cadastro", "acao": "cadastro", "x": 68.0, "y": 67.0, "w": 13.0, "h": 10.0}, {"chave": "admin", "rotulo": "Criar Administrador", "acao": "admin", "x": 68.0, "y": 77.0, "w": 13.0, "h": 10.0},] def _carregar_zonas_home(): """Zonas salvas no banco (com fallback nos padrões).""" zonas = _zonas_home_default() linhas = [] try: conexao = sqlite3.connect(DB_FILE) linhas = conexao.execute("SELECT chave, valor FROM configuracoes").fetchall() conexao.close() except Exception: pass por_chave = {z["chave"]: z for z in zonas} for chave, valor in linhas: if not valor or not chave.startswith("home_zona_"): continue k = chave[len("home_zona_"):] if k not in por_chave: continue partes = valor.split(";") if len(partes) != 4: continue try: x, y, w, h = (float(p.strip()) for p in partes) except ValueError: continue z = por_chave[k] z["x"], z["y"], z["w"], z["h"] = x, y, w, h return zonas def _salvar_zonas_home(zonas): """Persiste as zonas ajustadas na tabela configuracoes.""" conexao = None try: conexao = sqlite3.connect(DB_FILE) for z in zonas: conexao.execute("INSERT INTO configuracoes (chave, valor) VALUES (?, ?) " "ON CONFLICT(chave) DO UPDATE SET valor = excluded.valor", (f"home_zona_{z['chave']}", f"{z['x']};{z['y']};{z['w']};{z['h']}",),) conexao.commit() finally: if conexao: conexao.close() def _montar_html_mascaras(b64, zonas, ajuste=False, valores=None): """Monta a imagem da home com as máscaras (links transparentes) sobre ela.""" partes = ["<style>" ".home-zona{position:absolute;cursor:pointer;z-index:5;border-radius:10px;}" ".home-zona:hover{box-shadow:0 0 0 3px rgba(255,255,255,.5)," "0 0 0 7px rgba(0,255,102,.4);}" ".home-ajuste{position:absolute;z-index:6;border:2px dashed #00FF66;" "background:rgba(0,255,102,.10);border-radius:10px;}" ".home-ajuste span{position:absolute;top:-1.3em;left:0;font-size:.72rem;" "color:#00FF66;background:rgba(0,0,0,.65);padding:1px 8px;border-radius:6px;" "white-

space:nowrap;}" "</style>", " ",] if b64: partes.append("<img src='%s' style='width:100%%;border-radius:14px;" "display:block;box-shadow:0 8px 30px rgba(0,0,0,.35);'>" % b64) for z in zonas: if valores and z["chave"] in valores: x, y, w, h = valores[z["chave"]] else: x, y, w, h = z["x"], z["y"], z["w"], z["h"] estilo = ("position:absolute;left:%.1f%%;top:%.1f%%;width:%.1f%%;height:%.1f%%" % (x, y, w, h)) if ajuste: partes.append(" " "%s " % (estilo, z["rotulo"])) else: partes.append("" % (z["acao"], estilo, z["rotulo"], z["rotulo"])) partes.append(" ") return "".join(partes) def _tratar_acao_home(): """Processa cliques das máscaras e do modo de ajuste (via query param)."""" _acao = st.query_params.get("acao") or st.session_state.pop("acao_home", None) _ajustar = st.query_params.get("ajustar") _sair = st.query_params.get("ajuste_sair") if _acao: if _acao == "acesso": st.session_state["home_tela"] = "acesso" elif _acao in ("cadastro", "login"): st.session_state["home_tela"] = "cadastro" elif _acao == "admin": st.session_state["home_tela"] = "criar_admin" st.session_state.pop("modo_ajuste", None) st.query_params.clear() st.rerun() return if _ajustar: st.session_state["modo_ajuste"] = True st.query_params.clear() st.rerun() return if _sair: st.session_state["modo_ajuste"] = False st.query_params.clear() st.rerun() def _render_home_fallback(): """Home sem imagem: botões textuais (fallback seguro)."""" st.markdown("<h1 style='text-align:center'>🔐 Cofre de Toxinas</h1>", unsafe_allow_html=True,) st.markdown(" Ministério do Meio Ambiente — " "controle de acesso ao cofre de segurança máxima. ", unsafe_allow_html=True,) st.divider() _sem_creds = not existe_usuario_com_credenciais() _cols = st.columns(3) if _sem_creds else st.columns(2) with _cols[0]: with st.container(border=True): st.markdown("### 📸 Cadastro") st.caption("Registre colaboradores, fotos e permissões. **Somente** " "colaboradores autorizados (❌ permissão de cadastro).") if st.button("Cadastrar", width="stretch", type="primary", key="home_cadastro"): st.session_state["home_tela"] = "cadastro" st.rerun() with _cols[1]: with st.container(border=True): st.markdown("### 🔐 Acesso") st.caption("Valide o acesso ao cofre por reconhecimento facial. " "Colaboradores com cadastro entram conforme o nível.") if st.button("Acessar", width="stretch", type="primary", key="home_acesso"): st.session_state["home_tela"] = "acesso" st.rerun() if _sem_creds: with _cols[2]: with st.container(border=True): st.markdown("### 🛡️ Primeiro Acesso") st.caption("Nenhuma credencial cadastrada: crie a conta do " "administrador do sistema (nível 3).") if st.button("Criar Administrador", width="stretch", type="secondary", key="home_admin"): st.session_state["home_tela"] =

"criar_admin" st.rerun() def _render_ajuste_mascaras(): """Modo de ajuste visual das máscaras sobre a imagem da home.""" st.title("⚙️ Ajuste da Home (áreas clicáveis)") st.caption("Use os controles abaixo para mover/redimensionar cada máscara até ela " "cobrir o elemento correspondente na imagem. Depois clique em " "**💾 Salvar posições**.") _img = _encontrar_imagem_home() _b64 = _imagem_home_base64(_img) if _img else None _sem_creds = not existe_usuario_com_credenciais() zonas = [z for z in _carregar_zonas_home() if z["chave"] != "admin" or _sem_creds] for z in zonas: _base = f"mask_{z['chave']}" st.session_state.setdefault(f"{_base}_x", z["x"]) st.session_state.setdefault(f"{_base}_y", z["y"]) st.session_state.setdefault(f"{_base}_w", z["w"]) st.session_state.setdefault(f"{_base}_h", z["h"]) valores = {} for z in zonas: _base = f"mask_{z['chave']}" valores[z["chave"]] = (st.session_state[f"{_base}_x"], st.session_state[f"{_base}_y"], st.session_state[f"{_base}_w"], st.session_state[f"{_base}_h"],) if _b64: st.markdown(_montar_html_mascaras(_b64, zonas, ajuste=True, valores=valores), unsafe_allow_html=True,) else: st.warning("⚠️ Nenhuma imagem encontrada em `IMAGEMHOME/`. As áreas de ajuste " "não têm referência visual.") st.divider() defaults = {z["chave"]: z for z in _zonas_home_default()} for u in zonas: _base = f"mask_{u['chave']}" with st.container(border=True): st.markdown(f"**{u['rotulo']}**") c1, c2, c3, c4 = st.columns(4) c1.slider("X (%)", 0.0, 100.0, key=f"{_base}_x", step=0.5) c2.slider("Y (%)", 0.0, 100.0, key=f"{_base}_y", step=0.5) c3.slider("Largura (%)", 0.0, 100.0, key=f"{_base}_w", step=0.5) c4.slider("Altura (%)", 0.0, 100.0, key=f"{_base}_h", step=0.5) if st.button("↺ Restaurar padrão desta zona", key=f"{_base}_reset"): st.session_state[f"{_base}_x"] = defaults[u["chave"]]["x"] st.session_state[f"{_base}_y"] = defaults[u["chave"]]["y"] st.session_state[f"{_base}_w"] = defaults[u["chave"]]["w"] st.session_state[f"{_base}_h"] = defaults[u["chave"]]["h"] st.rerun() c1, c2, c3 = st.columns(3) with c1: if st.button("💾 Salvar posições", type="primary", width="stretch"): _novas = [] for u in zonas: _base = f"mask_{u['chave']}" _novas.append({ "chave": u["chave"], "rotulo": u["rotulo"], "acao": u["acao"], "x": float(st.session_state[f"{_base}_x"]), "y": float(st.session_state[f"{_base}_y"]), "w": float(st.session_state[f"{_base}_w"]), "h": float(st.session_state[f"{_base}_h"]), }) _salvar_zonas_home(_novas) st.success("✅ Posições salvas!") with c2: if st.button("↩️ Restaurar padrão e salvar", width="stretch"): _novas = [] for u in zonas: p = defaults[u["chave"]] _novas.append({ "chave": u["chave"], "rotulo": u["rotulo"], "acao": u["acao"], "x": p["x"], "y": p["y"], "w": p["w"], "h": p["h"], }) _salvar_zonas_home(_novas) st.success("✅ Padrão restaurado e salvo!") with c3: if st.button("✅ Concluir", width="stretch"): st.session_state["modo_ajuste"] = False st.rerun() def _render_home(): """Tela inicial (pré-login) com máscaras clicáveis sobre a imagem."""

st.markdown(HOME_CSS, unsafe_allow_html=True) _tratar_acao_home() if st.session_state.get("modo_ajuste"): _render_ajuste_mascaras() return _img = _encontrar_imagem_home() if not _img: _render_home_fallback() return _b64 = _imagem_home_base64(_img) _sem_creds = not existe_usuario_com_credenciais() zonas = [z for z in _carregar_zonas_home() if z["chave"] != "admin" or _sem_creds] st.markdown(_montar_html_mascaras(_b64, zonas), unsafe_allow_html=True) st.markdown(" " "Clique sobre os elementos da tela para acessar o sistema. ", unsafe_allow_html=True,) st.caption("[⚙️ Ajustar áreas clicáveis](?ajustar=1) — " "para administradores; requer cuidado.") def _render_login(destino=None): """Tela de login por usuário/senha.""" if st.button("← Voltar ao Início", key="voltar_login"): st.session_state["home_tela"] = "home" st.rerun() st.markdown("<h1 style='text-align:center'>🔐 Cofre de Toxinas — Ministério do Meio Ambiente</h1>", unsafe_allow_html=True,) st.markdown(" Acesso restrito — identifique-se " "para continuar. ", unsafe_allow_html=True,) st.divider() with st.form("login_credenciais", clear_on_submit=False): usuario = st.text_input("Usuário:", key="login_usuario") senha = st.text_input("Senha:", type="password", key="login_senha") enviar = st.form_submit_button("🔓 Entrar no Sistema", type="primary") if enviar: ok, msg = _autenticar({"login": usuario, "senha": senha}) if ok: _aut = st.session_state.get("usuario_logado", {}) if destino == "cadastro" and not _aut.get("pode_cadastrar"): st.session_state["home_aviso"] = ("⚠️ Você não possui **permissão de cadastro**. Ela é " "concedida pelo nível 3 na aba 🛡️ Gestão de Acessos.") st.session_state["redirecionar_para"] = "🔐 Cofre de Toxinas do MMA" elif destino == "cadastro": st.session_state["redirecionar_para"] = "📸 Cadastro de Colaboradores" else: st.session_state["redirecionar_para"] = "🔐 Cofre de Toxinas do MMA" st.rerun() else: st.error(f"🚫 **Acesso negado.** {msg}") def _render_criar_admin(): """Primeira execução: cria o usuário administrador inicial (nível 3).""" if st.button("← Voltar ao Início", key="voltar_admin"): st.session_state["home_tela"] = "home" st.rerun() st.markdown("<h1 style='text-align:center'>🛡️ Criar Primeiro Administrador</h1>", unsafe_allow_html=True,) st.markdown(" Nenhum usuário com credenciais " "foi encontrado. Crie a conta inicial do administrador do sistema " "(nível 3 — Ministro, com permissão de cadastro). ", unsafe_allow_html=True,) st.divider() with st.form("criar_admin_form", clear_on_submit=False): nome = st.text_input("Nome Completo do Administrador:*", key="adm_nome") cargo = st.text_input("Cargo / Função:", key="adm_cargo") usuario = st.text_input("Usuário (login):*", key="adm_usuario") senha = st.text_input("Senha:*",

type="password", key="adm_senha") senha2 = st.text_input("Confirmar Senha:*", type="password", key="adm_senha2") criar = st.form_submit_button("✅ Criar Administrador", type="primary") if criar: nome = nome.strip() usuario = (usuario or "").strip() if not nome or not usuario: st.error("⚠️ Preencha Nome Completo e Usuário.") return if len(senha) < 6: st.error("⚠️ A senha deve ter ao menos 6 caracteres.") return if senha != senha2: st.error("⚠️ As senhas não conferem.") return try: conn = sqlite3.connect(DB_FILE) cur = conn.cursor() cur.execute("SELECT COUNT(*) FROM colaboradores WHERE usuario = ? COLLATE NOCASE", (usuario,),) if cur.fetchone()[0] > 0: conn.close() st.error("⚠️ Este usuário já está em uso.") return senha_hash = hash_senha(senha) data_hoje = datetime.now().strftime("%Y-%m-%d") cur.execute("INSERT INTO colaboradores " "(nome, cargo, divisao, nivel_acesso, usuario, senha_hash, pode_cadastrar, " "foto_path, data_cadastro) " "VALUES (?, ?, 'Sem divisão', 3, ?, ?, 1, '', ?)", (nome, cargo, usuario, senha_hash, data_hoje),) novo_id = cur.lastrowid conn.commit() conn.close() st.session_state.pop("sessao_expirada", None) st.session_state["usuario_logado"] = { "id": novo_id, "nome": nome, "cargo": cargo, "divisao": "Sem divisão", "nivel_acesso": 3, "usuario": usuario, "pode_cadastrar": 1, "login_ts": time.time(), "metodo": "credenciais", } st.session_state["autenticado"] = True st.success(f"✅ Administrador **{nome}** criado e autenticado! " "Você já pode cadastrar os demais colaboradores.") st.rerun() except Exception as e: registrar_erro("AUTENTICACAO", "Criar Administrador", str(e)) st.error(f"❌ Erro ao criar o administrador. Veja a aba 🐞 Registro de " f"Erros para ajuda. ({e})") def _encerrar_sessao(): st.session_state.pop("usuario_logado", None) st.session_state.pop("autenticado", None) st.session_state.pop("sessao_expirada", None) st.session_state.pop("live_frames", None) st.session_state.pop("redirecionar_para", None) st.session_state.pop("home_aviso", None) st.session_state["home_tela"] = "home" def _aba_simulacao(): """Validação biométrica / login ao cofre (acessível com ou sem credenciais).""" st.markdown(MOLDURA_CIRCULAR_CSS, unsafe_allow_html=True) st.title("📹 Validação Biométrica / Login ao Cofre de Toxinas") if not _sessao_valida(): st.info("Registro facial reconhecido inicia sua sessão no sistema " "(colaboradores cadastrados antes do novo esquema de credenciais " "ainda podem acessar por aqui).") nivel_requerido = st.selectbox("Selecione o nível do compartimento do cofre sendo acessado:", options=list(NIVEIS_ACESSO.keys()), format_func=rotulo_nivel,) area_requerida = rotulo_nivel(nivel_requerido) st.markdown(f"**Status da Câmera:** Monitorando o acesso ao compartimento " f"**[{area_requerida}]** do cofre.") bloq, minutos = _bloqueio_ativo(area_requerida) if bloq: st.error(f"⛔ **BLOQUEIO TEMPORÁRIO ATIVADO**\n\nO compartimento " f"**{area_requerida}** está com a

validação **suspensa por " f"~{minutos} min** após tentativas repetidas de fraude ou " f"acesso indevido foi registrado para auditoria.") tent_linhas = _tentativas_recentes(area_requerida) if tent_linhas: st.dataframe(pd.DataFrame(tent_linhas), width="stretch") return metodo_leitura = st.radio("Origem da imagem:", ["Tirar Foto pela Webcam", "🎥 Mini Vídeo Anti-Fraude (mova o rosto)",], key="metodo_leitura_radio",) if metodo_leitura == "🎥 Mini Vídeo Anti-Fraude (mova o rosto)": _render_mini_video(nivel_requerido, area_requerida) return input_test = st.camera_input("Centralize o rosto no círculo para leitura") if input_test is not None: test_bytes = input_test.getvalue() temp_test_path = "temp_test.jpg" with open(temp_test_path, "wb") as f: f.write(test_bytes) recortar_centro_rosto(temp_test_path) try: carregar_modelo_facial() with st.spinner("Validando biometria facial..."): melhor_candidato, menor_distancia = _melhor_match_biometrico(temp_test_path) if os.path.exists(temp_test_path): os.remove(temp_test_path) if melhor_candidato is not None and menor_distancia <= 0.35: colab_id, colab_nome = melhor_candidato _conceder_ou_negar(colab_id, colab_nome, menor_distancia, nivel_requerido, area_requerida,) else: st.error(f"🚨 **ACESSO NEGADO!** Biometria não bateu com nenhum cadastrado. (Menor distância calculada: {menor_distancia:.3f})") registrar_log("Desconhecido", area_requerida, "NEGADO", {"motivo": "Biometria não reconhecida (foto)"},) _registrar_e_verificar_bloqueio(area_requerida, "Biometria não reconhecida (foto).", origem="foto",) except Exception as e: registrar_erro("GERAL", "Validação de Acesso", str(e)) st.error(f"❌ Ocorreu um erro inesperado na validação de acesso. Veja " f"a aba 🐞 Registro de Erros para ajuda. ({e})") st.sidebar.title("🔐 Cofre de Toxinas") _sessao_ok = _sessao_valida() _usuario_sidebar = st.session_state.get("usuario_logado") if not _sessao_ok: if _usuario_sidebar and st.session_state.get("sessao_expirada"): st.sidebar.warning("⏱️ **Sessão expirada por inatividade.**\nFaça login novamente.") st.session_state.pop("sessao_expirada", None) else: st.sidebar.info("🔴Nenhuma sessão ativa.") st.sidebar.caption("🔒 Áreas sensíveis seguem as permissões por nível. Colaboradores " "cadastrados antes do novo esquema entram pela biometria.") if "home_tela" not in st.session_state: st.session_state["home_tela"] = "home" _tela = st.session_state["home_tela"] if _tela == "home": _render_home() elif _tela == "cadastro": _render_login(destino="cadastro") elif _tela == "acesso": if st.button("← Voltar ao Início", key="voltar_acesso"): st.session_state["home_tela"] = "home" st.rerun() st.divider() _aba_simulacao() else: _render_criar_admin() st.stop() # ---------- # USUÁRIO AUTENTICADO — Navegação e sessão # ---------- _opcoes_nav =

["🔐 Cofre de Toxinas do MMA", "📹 Validar Acesso / Login Biométrico", "📈 Dashboard Analítico de Acessos", "📊 Dashboard e Relatórios", "🐞 Registro de Erros / Logs", "📸 Cadastro de Colaboradores",] if usuario_tem_nivel(3): _opcoes_nav.insert(5, "🛡️ Gestão de Acessos") _destino_inicial = st.session_state.get("redirecionar_para") if _destino_inicial in _opcoes_nav: _idx_nav = _opcoes_nav.index(_destino_inicial) else: _idx_nav = 0 opcao = st.sidebar.radio("Navegação:", _opcoes_nav, index=_idx_nav,) st.session_state.pop("redirecionar_para", None) _usuario_sidebar = st.session_state.get("usuario_logado") if _usuario_sidebar: st.sidebar.success(f"🟢 **{_usuario_sidebar['nome']}**\n{rotulo_nivel(_usuario_sidebar['nivel_acesso'])}") if _usuario_sidebar.get("pode_cadastrar"): st.sidebar.caption("📝 Permissão de cadastro de colaboradores") if st.sidebar.button("🔒 Encerrar Sessão", use_container_width=True): _encerrar_sessao() st.rerun() st.sidebar.divider() if "home_aviso" in st.session_state: _aviso = st.session_state.pop("home_aviso") st.warning(_aviso) def ao_trocar_aba_guard(fn, modulo): """Envolve a lógica de uma aba em try/except para capturar erros inesperados.""" try: fn() except Exception as e: registrar_erro("GERAL", modulo, str(e)) st.error(f"❌ Ocorreu um erro inesperado nesta tela. Veja a aba 🐞 " f"Registro de Erros para ajuda. ({e})") if opcao == "📸 Cadastro de Colaboradores": def _aba_cadastro(): if not usuario_pode_cadastrar(): st.title("📸 Cadastro de Colaboradores") st.warning("🔒 **Acesso restrito.** Apenas colaboradores autorizados " "(com permissão de cadastro, definida na aba 🛡️ Gestão de " "Acessos pelo nível 3) podem cadastrar novos colaboradores.") return st.markdown(MOLDURA_CIRCULAR_CSS, unsafe_allow_html=True) st.title("📸 Cadastro de Colaboradores e Permissões") if "msg_status" in st.session_state: tipo, msg = st.session_state.pop("msg_status") if tipo == "success": st.success(msg) else: st.error(msg) col1, col2 = st.columns([1, 1]) with col1: st.subheader("Dados Pessoais") st.text_input("Nome Completo:*", key="reg_nome") st.text_input("Cargo / Função:", key="reg_cargo") st.selectbox("Divisão do Ministério:", options=["Sem divisão"] + DIVISOES_MMA, key="reg_divisao", help=("Diretores (nível 2) e o ministro (nível 3) pertencem a " "divisões específicas do MMA."),) st.selectbox("Nível de Acesso ao Cofre:*", options=list(NIVEIS_ACESSO.keys()), format_func=rotulo_nivel, key="reg_nivel", help=("Níveis **cumulativos**: um colaborador de nível N acessa " "tudo dos níveis anteriores (1 a N)."),) st.caption(descricao_nivel(st.session_state.get("reg_nivel", 1))) st.divider() st.subheader("Credenciais de Acesso") st.text_input("Usuário (login):*", key="reg_usuario", placeholder="ex.: maria.silva", help="Este usuário será usado para entrar no sistema.",) st.text_input("Senha:*", type="password", key="reg_senha", help="Senha inicial (mínimo 6 caracteres).",) with col2: st.subheader("Captura da Foto")

metodo_foto = st.radio("Origem da imagem:", ["Tirar Foto pela Webcam", "Fazer Upload de Imagem"], key="metodo_foto_radio",) if metodo_foto == "Tirar Foto pela Webcam": foto_input = st.camera_input("Centralize o rosto no círculo verde antes de tirar a foto") else: foto_input = st.file_uploader("Escolha um arquivo JPG/PNG", type=["jpg", "jpeg", "png"]) if foto_input is not None: st.session_state["foto_capturada_bytes"] = foto_input.getvalue() st.button("💾 Cadastrar Colaborador", type="primary", on_click=salvar_colaborador,) st.divider() st.subheader("👥 Colaboradores Já Cadastrados") try: conn = sqlite3.connect(DB_FILE) df_colabs = pd.read_sql_query("SELECT id, nome, cargo, divisao, nivel_acesso, usuario, " "pode_cadastrar, data_cadastro FROM colaboradores", conn,) if not df_colabs.empty: df_colabs["nivel"] = df_colabs["nivel_acesso"].map(rotulo_nivel) df_colabs["pde_cadastrar"] = df_colabs["pode_cadastrar"].map({1: "✅ Sim", 0: "—"}) st.dataframe(df_colabs[["id", "nome", "cargo", "divisao", "nivel", "usuario", "pde_cadastrar", "data_cadastro",]].rename(columns={ "id": "ID", "nome": "Nome", "cargo": "Cargo", "divisao": "Divisão", "nivel": "Nível de Acesso", "usuario": "Usuário", "pde_cadastrar": "Pode Cadastrar", "data_cadastro": "Data Cadastro", }), width="stretch",) else: st.info("Nenhum colaborador cadastrado ainda.") conn.close() except Exception as e: registrar_erro("BANCO", "Lista de Colaboradores", str(e)) st.error(f"Erro ao carregar colaboradores. Veja a aba 🐞 Registro de Erros " f"para ajuda. ({e})") ao_trocar_aba_guard(_aba_cadastro, "Cadastro de Colaboradores") elif opcao == "📹 Validar Acesso / Login Biométrico": ao_trocar_aba_guard(_aba_simulacao, "Validar Acesso / Login Biométrico") elif opcao == "📈 Dashboard Analítico de Acessos": def _aba_dash_analitico(): if not usuario_tem_nivel(2): st.title("📈 Dashboard Analítico de Acessos") st.warning("🔒 **Acesso restrito.** O dashboard analítico de acessos só é " "liberado para colaboradores de **nível 2 ou 3** (Diretores de " "Divisões Específicas e Ministro do Meio Ambiente).") return st.title("📈 Dashboard Analítico de Acessos ao Cofre") st.caption("Visão consolidada dos eventos de acesso ao **Cofre de Toxinas**: " "tendências temporais, distribuição por compartimento do cofre, " "colaboradores mais ativos e diagnóstico de segurança.") df_logs = _preparar_logs() if df_logs.empty: st.info("Nenhum registro de acesso encontrado.") return data_min = df_logs["data"].min() data_max = df_logs["data"].max() st.divider() st.subheader("🔎 Filtros") c1, c2 = st.columns([1, 1]) with c1: periodo = st.selectbox("Período rápido:", ["Tudo", "Hoje", "Últimos 7 dias", "Últimos 30 dias"],) with c2: data_range = st.date_input("Intervalo de datas:", value=(data_min, data_max), min_value=data_min, max_value=data_max,) compartimentos_opcoes = sorted(df_logs["area_dados"].unique(), key=lambda s: (int(s.split("–")[0].replace("Nível", "").strip()) if "Nível" in s else 99),) f1, f2, f3 =

st.columns(3) with f1: area_filter = st.multiselect("Filtrar por Compartimento do Cofre:", options=compartimentos_opcoes, default=compartimentos_opcoes,) with f2: status_filter = st.multiselect("Filtrar por Status:", options=sorted(df_logs["status"].unique()), default=sorted(df_logs["status"].unique()),) with f3: nome_filter = st.text_input("Buscar por Colaborador:") if len(data_range) == 2: data_inicio, data_fim = data_range else: data_inicio, data_fim = data_min, data_max df = df_logs[(df_logs["data"] >= data_inicio) & (df_logs["data"] <= data_fim) & (df_logs["area_dados"].isin(area_filter)) & (df_logs["status"].isin(status_filter))] if nome_filter: df = df[df["colaborador_nome"].str.contains(nome_filter, case=False, na=False)] if df.empty: st.warning("Nenhum evento no período/filtros selecionados.") return total = len(df) permitidos = len(df[df["status"] == "PERMITIDO"]) negados = len(df[df["status"] == "NEGADO"]) taxa = (permitidos / total * 100) if total else 0.0 dias_periodo = max((data_fim - data_inicio).days, 1) media_dia = total / dias_periodo m1, m2, m3, m4, m5 = st.columns(5) m1.metric("Total de Acessos", total) m2.metric("Permitidos", permitidos) m3.metric("Negados", negados) m4.metric("Taxa de Aprovação", f"{taxa:.1f}%") m5.metric("Média / dia", f"{media_dia:.1f}") st.divider() st.subheader("📅 Tendências Temporais") serie_dia = (df.groupby(["data", "status"], as_index=False) .size() .rename(columns={"size": "contagem"})) c1, c2 = st.columns([2, 1]) with c1: chart_linha = (alt.Chart(serie_dia) .mark_line(point=True, strokeWidth=2) .encode(x=alt.X("data:T", title="Data"), y=alt.Y("contagem:Q", title="Nº de acessos"), color=alt.Color("status:N", title="Status", scale=alt.Scale(domain=["PERMITIDO", "NEGADO"], range=["#00C853", "#D50000"],),), tooltip=["data:T", "status:N", "contagem:Q"],) .properties(height=320)) st.altair_chart(chart_linha, use_container_width=True) with c2: status_tot = (df.groupby("status", as_index=False) .size() .rename(columns={"size": "contagem"})) chart_status = (alt.Chart(status_tot) .mark_arc() .encode(theta=alt.Theta("contagem:Q", stack=True), color=alt.Color("status:N", scale=alt.Scale(domain=["PERMITIDO", "NEGADO"], range=["#00C853", "#D50000"],),), tooltip=["status:N", "contagem:Q"],) .properties(height=320)) st.altair_chart(chart_status, use_container_width=True) st.divider() st.subheader("🏢 Distribuição por Compartimento do Cofre") dist_area = (df.groupby(["compartimento_curto", "nivel"], as_index=False) .size() .rename(columns={"size": "contagem"}) .sort_values(["nivel", "compartimento_curto"])) chart_area = (alt.Chart(dist_area) .mark_bar() .encode(x=alt.X("contagem:Q", title="Nº de eventos"), y=alt.Y("compartimento_curto:N", title="", sort="-x"), color=alt.Color("nivel:O", title="Nível de segurança", scale=alt.Scale(domain=["1", "2", "3"], range=["#2E7D32", "#F9A825",

"#C62828"],),), tooltip=["compartimento_curto:N", "nivel:O", "contagem:Q"],) .properties(height=320)) st.altair_chart(chart_area, use_container_width=True) st.caption("Níveis cumulativos do cofre de toxinas — " "🟢 Nível 1: **Permissão Geral** · " "🟡 Nível 2: **Diretores de Divisões Específicas** · " "🔴 Nível 3: **Ministro do Meio Ambiente** (acesso exclusivo).") st.divider() st.subheader("🎖️ Equipe do Cofre") try: conn2 = sqlite3.connect(DB_FILE) df_colabs = pd.read_sql_query("SELECT id, nome, cargo, divisao, nivel_acesso " "FROM colaboradores", conn2,) conn2.close() if not df_colabs.empty: e1, e2, e3, e4 = st.columns(4) e1.metric("Colaboradores no cofre", len(df_colabs)) e2.metric("Nível 1 (Permissão Geral)", int((df_colabs["nivel_acesso"] == 1).sum())) e3.metric("Nível 2 (Diretores)", int((df_colabs["nivel_acesso"] == 2).sum())) e4.metric("Nível 3 (Ministro)", int((df_colabs["nivel_acesso"] == 3).sum())) df_colabs["nivel"] = df_colabs["nivel_acesso"].map(rotulo_nivel) st.dataframe(df_colabs[["nome", "cargo", "divisao", "nivel"]].rename(columns={ "nome": "Nome", "cargo": "Cargo", "divisao": "Divisão", "nivel": "Nível de Acesso", }), width="stretch",) else: st.info("Nenhum colaborador cadastrado no cofre ainda.") except Exception as e: st.caption(f"Equipe indisponível: {e}") st.divider() st.subheader("👥 Colaboradores Mais Ativos") top_colabs = (df.groupby("colaborador_nome", as_index=False) .size() .rename(columns={"size": "contagem"}) .sort_values("contagem", ascending=False) .head(10)) chart_top = (alt.Chart(top_colabs) .mark_bar() .encode(x=alt.X("contagem:Q", title="Nº de eventos"), y=alt.Y("colaborador_nome:N", title="", sort="-x"), color=alt.Color("contagem:Q", scale=alt.Scale(scheme="viridis")), tooltip=["colaborador_nome:N", "contagem:Q"],) .properties(height=300)) st.altair_chart(chart_top, use_container_width=True) st.divider() st.subheader("🕐 Movimentação por Hora do Dia") serie_hora = (df.groupby(["hora", "status"], as_index=False) .size() .rename(columns={"size": "contagem"})) chart_hora = (alt.Chart(serie_hora) .mark_bar() .encode(x=alt.X("hora:O", title="Hora do dia"), y=alt.Y("contagem:Q", title="Nº de eventos"), color=alt.Color("status:N", scale=alt.Scale(domain=["PERMITIDO", "NEGADO"], range=["#00C853", "#D50000"],),), tooltip=["hora:O", "status:N", "contagem:Q"],) .properties(height=300)) st.altair_chart(chart_hora, use_container_width=True) st.divider() st.subheader("🚨 Diagnóstico de Segurança do Cofre") negados_df = df[df["status"] == "NEGADO"] if not negados_df.empty: top_negados_area = (negados_df.groupby("area_dados") .size() .sort_values(ascending=False) .head(3)) st.warning(f"**{len(negados_df)} tentativas negadas** no período. " "Compartimentos mais visados: " + ", ".join(f"`{s}` ({n})" for s, n in top_negados_area.items()) + ".") desconhecidos = negados_df[negados_df["colaborador_nome"] == "Desconhecido"] if

not desconhecidos.empty: st.warning(f"**{len(desconhecidos)} eventos** de pessoa não " "identificada (possível tentativa de intrusão ao cofre).") tentativa_sem_perm = negados_df[negados_df["colaborador_nome"] != "Desconhecido"] if not tentativa_sem_perm.empty: st.warning(f"**{len(tentativa_sem_perm)} eventos** de colaboradores " "identificados sem permissão para o compartimento do cofre.") else: st.success("✅ Nenhuma tentativa negada no período selecionado.") st.divider() st.subheader("📄 Auditoria Detalhada dos Eventos") st.dataframe(df[["data_hora", "colaborador_nome", "compartimento_curto", "nivel", "status",]] .sort_values("data_hora", ascending=False) .rename(columns={ "data_hora": "Data / Hora", "colaborador_nome": "Colaborador", "compartimento_curto": "Compartimento", "nivel": "Nível", "status": "Status", }), width="stretch",) df_export = df.rename(columns={"area_dados": "compartimento"}).drop(columns=["compartimento_curto"]) csv_data = df_export.to_csv(index=False).encode("utf-8") st.download_button(label="📥 Exportar Eventos Filtrados (CSV)", data=csv_data, file_name=(f"eventos_cofre_{datetime.now().strftime('%Y%m%d')}.csv"), mime="text/csv",) st.divider() st.subheader("📄 Relatório PDF") st.caption("Gera um relatório executivo em PDF com os indicadores dos filtros " "ativos, gráficos ilustrativos e as tabelas de auditoria.") with st.spinner("Gerando relatório PDF com gráficos..."): pdf_bytes = gerar_relatorio_pdf(df) st.download_button(label="📄 Baixar Relatório PDF (Dados Filtrados + Gráficos)", data=pdf_bytes, file_name=(f"relatorio_cofre_{datetime.now().strftime('%Y%m%d_%H%M')}.pdf"), mime="application/pdf",) ao_trocar_aba_guard(_aba_dash_analitico, "Dashboard Analítico de Acessos") elif opcao == "📊 Dashboard e Relatórios": def _aba_dash_relatorios(): if not usuario_tem_nivel(2): st.title("📊 Dashboard e Relatórios") st.warning("🔒 **Acesso restrito.** O dashboard e relatórios de auditoria " "só é liberado para colaboradores de **nível 2 ou 3** " "(Diretores de Divisões Específicas e Ministro).") return st.title("📊 Auditoria e Métricas de Controle de Acesso ao Cofre") df_logs = _preparar_logs() if df_logs.empty: st.info("Nenhum registro de acesso encontrado.") return total_acessos = len(df_logs) permitidos = len(df_logs[df_logs["status"] == "PERMITIDO"]) negados = len(df_logs[df_logs["status"] == "NEGADO"]) m1, m2, m3, m4 = st.columns(4) m1.metric("Total de Acessos", total_acessos) m2.metric("Acessos Permitidos", permitidos) m3.metric("Tentativas Negadas", negados, delta_color="inverse" if negados > 0 else "normal",) m4.metric("Compartimentos monitorados", df_logs["nivel"].nunique()) st.divider() st.subheader("🔍 Filtros de Consulta") f1, f2, f3 = st.columns(3) compartimentos_opcoes =

sorted(df_logs["area_dados"].unique(), key=lambda s: (int(s.split("–")[0].replace("Nível", "").strip()) if "Nível" in s else 99),) with f1: area_filter = st.multiselect("Filtrar por Compartimento do Cofre:", options=compartimentos_opcoes, default=compartimentos_opcoes,) with f2: status_filter = st.multiselect("Filtrar por Status:", options=df_logs["status"].unique(), default=df_logs["status"].unique(),) with f3: nome_filter = st.text_input("Buscar por Colaborador:") df_filtered = df_logs[(df_logs["area_dados"].isin(area_filter)) & (df_logs["status"].isin(status_filter))] if nome_filter: df_filtered = df_filtered[df_filtered["colaborador_nome"].str.contains(nome_filter, case=False, na=False)] st.subheader("📄 Histórico de Acessos") st.dataframe(df_filtered[["data_hora", "colaborador_nome", "compartimento_curto", "nivel", "status",]] .rename(columns={ "data_hora": "Data / Hora", "colaborador_nome": "Colaborador", "compartimento_curto": "Compartimento", "nivel": "Nível", "status": "Status", }), width="stretch",) df_export = df_filtered.rename(columns={"area_dados": "compartimento"}).drop(columns=["compartimento_curto"]) csv_data = df_export.to_csv(index=False).encode("utf-8") st.download_button(label="📥 Exportar Relatório Filtrado (CSV)", data=csv_data, file_name=(f'relatorio_cofre_{datetime.now().strftime("%Y%m%d")}.csv'), mime="text/csv",) st.divider() st.subheader("📄 Relatório PDF") st.caption("Gera um relatório executivo completo em PDF com todos os " "indicadores, gráficos ilustrativos e as tabelas de auditoria.") with st.spinner("Gerando relatório PDF com gráficos..."): pdf_bytes = gerar_relatorio_pdf() st.download_button(label="📄 Baixar Relatório PDF (Completo + Gráficos)", data=pdf_bytes, file_name=(f"relatorio_cofre_completo_{datetime.now().strftime('%Y%m%d_%H%M')}.pdf"), mime="application/pdf",) ao_trocar_aba_guard(_aba_dash_relatorios, "Dashboard e Relatórios") elif opcao == "🐞 Registro de Erros / Logs": def _aba_logs(): if not usuario_tem_nivel(2): st.title("🐞 Registro de Erros / Logs") st.warning("🔒 **Acesso restrito.** O registro de erros/logs do sistema " "só é liberado para colaboradores de **nível 2 ou 3**.") return st.title("🐞 Registro de Erros / Logs") st.caption("Registros automáticos de erros capturados pela aplicação. Use os " "filtros para localizar um erro e veja abaixo como entender e resolver.") conn = sqlite3.connect(DB_FILE) df_erros = pd.read_sql_query("SELECT * FROM logs_erros", conn) conn.close() if df_erros.empty: st.info("Nenhum erro registrado até o momento. 🎉") return df_erros["data_hora"] = pd.to_datetime(df_erros["data_hora"]) df_erros["data"] = df_erros["data_hora"].dt.date st.divider() st.subheader("🔎 Filtros de Erros") f1, f2, f3 = st.columns(3) with f1: tipo_filter = st.multiselect("Filtrar por Tipo de Erro:", options=sorted(df_erros["tipo"].unique()),

default=sorted(df_erros["tipo"].unique()),) with f2: modulo_filter = st.multiselect("Filtrar por Módulo:", options=sorted(df_erros["modulo"].unique()), default=sorted(df_erros["modulo"].unique()),) with f3: busca_filter = st.text_input("Buscar na mensagem:") data_min = df_erros["data"].min() data_max = df_erros["data"].max() data_range = st.date_input("Intervalo de datas:", value=(data_min, data_max), min_value=data_min, max_value=data_max,) if len(data_range) == 2: data_inicio, data_fim = data_range else: data_inicio, data_fim = data_min, data_max df = df_erros[(df_erros["data"] >= data_inicio) & (df_erros["data"] <= data_fim) & (df_erros["tipo"].isin(tipo_filter)) & (df_erros["modulo"].isin(modulo_filter))] if busca_filter: df = df[df["mensagem"].str.contains(busca_filter, case=False, na=False)] st.divider() st.subheader("📊 Resumo") c1, c2, c3 = st.columns(3) c1.metric("Total de Erros (filtrado)", len(df)) if not df.empty: tipo_counts = (df.groupby("tipo", as_index=False) .size() .rename(columns={"size": "contagem"})) c2.metric("Tipos Distintos", len(tipo_counts)) mais_frequente = (tipo_counts.sort_values("contagem", ascending=False).iloc[0]) c3.metric("Erro mais Frequente", str(mais_frequente["tipo"])) st.write("**Distribuição por tipo de erro:**") chart_tipos = (alt.Chart(tipo_counts) .mark_bar() .encode(x=alt.X("tipo:N", title="Tipo"), y=alt.Y("contagem:Q", title="Nº de erros"), color=alt.Color("tipo:N", legend=None), tooltip=["tipo:N", "contagem:Q"],) .properties(height=250)) st.altair_chart(chart_tipos, use_container_width=True) st.divider() st.subheader("🗒️ Lista de Erros Registrados") if df.empty: st.warning("Nenhum erro atende aos filtros selecionados.") return st.dataframe(df[["data_hora", "tipo", "modulo", "mensagem"]].sort_values("data_hora", ascending=False), width="stretch",) st.divider() st.subheader("❓ Selecione um erro para ver a Ajuda / Como Resolver") opcoes = { f"{r['data_hora']} | {r['tipo']} | {r['modulo']}": r["id"] for _, r in df.sort_values("data_hora", ascending=False).iterrows() } escolha = st.selectbox("Erros (data/hora | tipo | módulo):", options=list(opcoes.keys()),) escolhido_id = opcoes[escolha] linha = df[df["id"] == escolhido_id].iloc[0] with st.expander("🔑 Entender o erro (em linguagem simples)", expanded=True): st.write(f"**Explicação:** {linha['descricao']}") with st.expander("🛠️ Como resolver / Sugestão", expanded=True): st.write(f"**Sugestão:** {linha['sugerida']}") with st.expander("🖥️etalhe técnico", expanded=False): st.code(linha["mensagem"], language="text") csv_data = df.to_csv(index=False).encode("utf-8") st.download_button(label="📥 Exportar Logs Filtrados (CSV)", data=csv_data, file_name=(f"logs_erros_{datetime.now().strftime('%Y%m%d')}.csv"), mime="text/csv",) st.divider() if st.button("🧹 Limpar todos os logs de erro", type="secondary", help="Remove permanentemente todos os registros de erro.",): limpar_logs_erros()

st.rerun() ao_trocar_aba_guard(_aba_logs, "Registro de Erros / Logs") elif opcao == "🛡️ Gestão de Acessos": def _aba_gestao_acessos(): if not usuario_tem_nivel(3): st.title("🛡️ Gestão de Acessos") st.warning("🔒 **Acesso restrito ao nível 3** — Ministro do Meio Ambiente.") return st.title("🛡️ Gestão de Acessos e Permissões") st.caption("Gerencie as credenciais (usuário/senha), o nível de acesso e a " "permissão de cadastro dos colaboradores. A permissão de cadastro " "deve ser concedida a apenas 1 ou 2 colaboradores de confiança.") try: conn = sqlite3.connect(DB_FILE) df_colabs = pd.read_sql_query("SELECT id, nome, cargo, divisao, nivel_acesso, usuario, " "pode_cadastrar, foto_path, data_cadastro FROM colaboradores " "ORDER BY nome", conn,) conn.close() df_colabs["usuario"] = df_colabs["usuario"].fillna("").astype(str) except Exception as e: registrar_erro("BANCO", "Gestão de Acessos", str(e)) st.error(f"Erro ao carregar colaboradores. Veja a aba 🐞 Registro de " f"Erros para ajuda. ({e})") return if df_colabs.empty: st.info("Nenhum colaborador cadastrado ainda.") return df_show = df_colabs.copy() df_show["nivel"] = df_show["nivel_acesso"].map(rotulo_nivel) df_show["pode_cadastrar_txt"] = df_show["pode_cadastrar"].map({1: "✅ Sim", 0: "—"}) df_show["credenciais"] = df_show["usuario"].fillna("—").replace("", "—") st.dataframe(df_show[["id", "nome", "cargo", "divisao", "nivel", "credenciais", "pode_cadastrar_txt"]].rename(columns={ "id": "ID", "nome": "Nome", "cargo": "Cargo", "divisao": "Divisão", "nivel": "Nível de Acesso", "credenciais": "Usuário", "pode_cadastrar_txt": "Pode Cadastrar", }), width="stretch",) st.divider() st.subheader("✏️ Editar Colaborador") opcoes = { f"{r['nome']} (ID {r['id']})": r["id"] for _, r in df_colabs.iterrows() } escolha = st.selectbox("Selecione o colaborador:", options=list(opcoes.keys()), key="gestao_escolha",) colab_id = opcoes[escolha] linha = df_colabs[df_colabs["id"] == colab_id].iloc[0] with st.form("gestao_editar"): novo_usuario = st.text_input("Usuário (login):", value=linha.get("usuario") or "", key="gestao_usuario",) nova_senha = st.text_input("Nova Senha (deixe em branco para manter):", type="password", key="gestao_senha",) novo_nivel = st.selectbox("Nível de Acesso:", options=list(NIVEIS_ACESSO.keys()), format_func=rotulo_nivel, index=int(linha["nivel_acesso"]) - 1, key="gestao_nivel",) pode_cadastrar = st.checkbox("Permitir que este colaborador cadastre novos colaboradores", value=bool(linha.get("pode_cadastrar")), key="gestao_pode_cadastrar", help="Conceda esta permissão a apenas 1 ou 2 colaboradores de confiança.",) salvar = st.form_submit_button("💾 Salvar Alterações", type="primary") if salvar: novo_usuario = (novo_usuario or "").strip() if not novo_usuario: st.error("⚠️ O usuário não pode ficar vazio.") return try: conn = sqlite3.connect(DB_FILE) cur = conn.cursor() cur.execute("SELECT COUNT(*) FROM colaboradores " "WHERE usuario = ? COLLATE

NOCASE AND id <> ?", (novo_usuario, colab_id),) if cur.fetchone()[0] > 0: conn.close() st.error("⚠️ Este usuário já está em uso por outro colaborador.") return if nova_senha: if len(nova_senha) < 6: conn.close() st.error("⚠️ A nova senha deve ter ao menos 6 caracteres.") return senha_hash = hash_senha(nova_senha) cur.execute("UPDATE colaboradores SET usuario = ?, senha_hash = ? " "WHERE id = ?", (novo_usuario, senha_hash, colab_id),) else: cur.execute("UPDATE colaboradores SET usuario = ? WHERE id = ?", (novo_usuario, colab_id),) cur.execute("UPDATE colaboradores SET nivel_acesso = ?, pode_cadastrar = ? " "WHERE id = ?", (novo_nivel, 1 if pode_cadastrar else 0, colab_id),) conn.commit() conn.close() user_atual = st.session_state.get("usuario_logado") if user_atual and user_atual.get("id") == colab_id: user_atual["nivel_acesso"] = novo_nivel user_atual["pode_cadastrar"] = 1 if pode_cadastrar else 0 user_atual["usuario"] = novo_usuario st.session_state["usuario_logado"] = user_atual st.success(f"✅ Alterações salvas para **{linha['nome']}**. " f"Nível: {rotulo_nivel(novo_nivel)}. " f"Pode cadastrar: {'✅ Sim' if pode_cadastrar else '—'}.") st.rerun() except Exception as e: registrar_erro("BANCO", "Gestão de Acessos (Editar)", str(e)) st.error(f"❌ Erro ao salvar alterações. Veja a aba 🐞 Registro de " f"Erros para ajuda. ({e})") ao_trocar_aba_guard(_aba_gestao_acessos, "Gestão de Acessos") elif opcao == "🔐 Cofre de Toxinas do MMA": def _aba_dados_geo(): st.title("🔐 Cofre de Toxinas — Ministério do Meio Ambiente") st.caption("Painel do cofre de segurança máxima que guarda informações sobre " "o uso de toxinas. O acesso a cada compartimento é controlado pelo " "nível do colaborador autenticado (níveis cumulativos).") usuario = st.session_state.get("usuario_logado") if not _sessao_valida(): expirou = st.session_state.get("sessao_expirada", False) st.warning("⏱️ **Sessão expirada ou nenhuma sessão ativa.** " if expirou else "⚠️ **Nenhuma sessão ativa.** " + "Faça login com **usuário e senha** para visualizar o cofre. " "Use a aba 📹 **Validar Acesso / Login Biométrico** para " "liberar compartimentos conforme o seu nível.") return st.success(f"🟢 Sessão ativa: **{usuario['nome']}** — {rotulo_nivel(usuario['nivel_acesso'])}") df_toxinas = carregar_dados_toxinas() # ---------- NÍVEL 1 ---------- st.divider() st.subheader("🗂️ 1. Catálogo Público de Toxinas (Permissão Geral)") st.caption(descricao_nivel(1)) if not usuario_tem_nivel(1): st.info("🔒 **Bloqueado para o seu nível.** O catálogo exige nível 1.") else: cols_catalogo = ["id_toxina", "nome", "classe", "tipo_acao", "periculosidade", "uso", "divisao_responsavel",] st.dataframe(df_toxinas[cols_catalogo], width="stretch") st.caption(f"Total de **{len(df_toxinas)}** toxinas registradas no catálogo do cofre.") st.markdown("#### 🛡️ Missão da Sala do Cofre") st.markdown("Este cofre, oculto nas profundezas do **Ministério do Meio Ambiente**, " "guarda os segredos mais sensíveis

sobre o uso de toxinas. As substâncias " "registradas aqui são tão poderosas que, se manuseadas de forma inadequada, " "poderiam provocar uma nova pandemia mundial, devastar ecossistemas e " "colocar em risco a vida na Terra. **Apenas pessoas devidamente " "autorizadas** têm acesso, em um esquema de permissões dividido em " "três níveis de segurança.") # ---------- NÍVEL 2 ---------- st.divider() st.subheader("🗺️ 2. Inventário e Localização (Diretores de Divisões)") st.caption(descricao_nivel(2)) if not usuario_tem_nivel(2): st.info("🔒 **Bloqueado para o seu nível.** O inventário exige nível 2.") else: c1, c2 = st.columns([2, 1]) with c1: chart_classe = (alt.Chart(df_toxinas) .mark_bar() .encode(x=alt.X("classe:N", title="", sort="-y"), y=alt.Y("estoque_unidades:Q", title="Unidades em estoque"), color=alt.Color("estado_conservacao:N", title="Estado"), tooltip=["classe:N", "estoque_unidades:Q", "estado_conservacao:N",],) .properties(height=300)) st.altair_chart(chart_classe, use_container_width=True) with c2: chart_comp = (alt.Chart(df_toxinas) .mark_circle(size=90) .encode(x=alt.X("compartimento:N", title=""), y=alt.Y("periculosidade:Q", title="Periculosidade (1-5)"), size=alt.Size("estoque_unidades:Q", title="Estoque"), color=alt.Color("classe:N", title="Classe"), tooltip=["id_toxina:N", "nome:N", "compartimento:N", "periculosidade:Q",],) .properties(height=300)) st.altair_chart(chart_comp, use_container_width=True) st.caption("Distribuição simulada das toxinas entre os compartimentos do cofre.") st.markdown("#### 🗄️ Estoque e conservação") cols_estoque = ["id_toxina", "nome", "compartimento", "divisao_responsavel", "estoque_unidades", "temperatura_c", "data_validade", "estado_conservacao",] st.dataframe(df_toxinas[cols_estoque].sort_values("estoque_unidades", ascending=False), width="stretch",) csv_inventario = df_toxinas.to_csv(index=False).encode("utf-8") st.download_button("📥 Exportar Inventário (CSV)", data=csv_inventario, file_name="inventario_toxinas.csv", mime="text/csv",) # ---------- NÍVEL 3 ---------- st.divider() st.subheader("🧪 3. Dossiê Completo e Movimentação (Ministro do Meio Ambiente)") st.caption(descricao_nivel(3)) if not usuario_tem_nivel(3): st.info("🔒 **Bloqueado para o seu nível.** O dossiê exige nível 3.") else: df_serie = serie_temporal_movimentacao() divisoes = list(df_serie["divisao"].unique()) selecao = st.multiselect("Divisões do Ministério:", options=divisoes, default=divisoes,) if selecao: df_serie = df_serie[df_serie["divisao"].isin(selecao)] chart_serie = (alt.Chart(df_serie) .mark_line(point=True) .encode(x=alt.X("mes:T", title="Mês / Ano"), y=alt.Y("movimentacoes:Q", title="Movimentações"), color=alt.Color("divisao:N", title="Divisão"), tooltip=["divisao:N", "mes:T", "movimentacoes:Q"],) .properties(height=320)) st.altair_chart(chart_serie,

use_container_width=True) st.caption("Série temporal simulada de movimentações de toxinas por divisão.") st.markdown("#### 🕵️ Cadeia de custódia (exemplo)") custodia = pd.DataFrame([{"lote": "TOX-0006/2026", "toxina": "Toxina Botulínica Tipo A", "de": "Compartimento A-04", "para": "Laboratório Central", "responsavel": "Diretor de Pesquisa", "data_hora": "2026-08-14 09:32"}, {"lote": "TOX-0001/2026", "toxina": "Ricina", "de": "Compartimento A-01", "para": "Sala de Análises", "responsavel": "Diretor de Toxicologia", "data_hora": "2026-08-15 14:05"}, {"lote": "TOX-0004/2026", "toxina": "Microcistina-LR", "de": "Compartimento B-01", "para": "Incinerador controlado", "responsavel": "Ministro do Meio Ambiente", "data_hora": "2026-09-02 10:12"},]) st.dataframe(custodia, width="stretch") # ---------- GOVERNANÇA ---------- st.divider() st.subheader("🏛️ Gestão e Auditoria do Cofre") st.caption("Painel executivo de governança, acessível pelo nível 3 (ministro).") if not usuario_tem_nivel(3): st.info("🔒 **Bloqueado para o seu nível.** A governança exige nível 3.") else: conn = sqlite3.connect(DB_FILE) df_colabs = pd.read_sql_query("SELECT id, nome, cargo, divisao, nivel_acesso, data_cadastro " "FROM colaboradores", conn,) df_logs_area = pd.read_sql_query("SELECT * FROM logs_acesso", conn) conn.close() c1, c2, c3, c4 = st.columns(4) c1.metric("Toxinas custodiadas", len(df_toxinas)) c2.metric("Unidades em estoque", int(df_toxinas["estoque_unidades"].sum())) c3.metric("Colaboradores cadastrados", len(df_colabs)) c4.metric("Eventos de acesso registrados", len(df_logs_area)) criticas = (df_toxinas["estado_conservacao"] == "Crítico").sum() if criticas: st.warning(f"⚠️ {criticas} toxina(s) em estado de conservação **CRÍTICO** " "exigem inspeção imediata.") else: st.success("✅ Todas as toxinas em estado de conservação estável.") if not df_colabs.empty: df_colabs["nivel"] = df_colabs["nivel_acesso"].map(rotulo_nivel) st.dataframe(df_colabs[["id", "nome", "cargo", "divisao", "nivel", "data_cadastro"]], width="stretch",) csv_full = df_toxinas.to_csv(index=False).encode("utf-8") st.download_button("📥 Exportar Base do Cofre (CSV)", data=csv_full, file_name="dados_cofre_toxinas.csv", mime="text/csv",) ao_trocar_aba_guard(_aba_dados_geo, "Cofre de Toxinas (Dados)")

Anexo B — Script auxiliar: gerar_excel.py

Gera a planilha `Relatorio_Controle_Acesso_Facial.xlsx` com abas "Dashboard & Métricas", "Logs de Acesso" e "Cadastro Colaboradores", utilizando `openpyxl` com fórmulas (COUNTIF/COUNTIFS), formatação condicional e paleta corporativa.

import openpyxl from openpyxl.styles import Alignment, Border, Font, PatternFill, Side

```
from openpyxl.utils import get_column_letter # Criação da pasta de trabalho e abas wb = openpyxl.Workbook() ws_dash = wb.active ws_dash.title = "Dashboard & Métricas" ws_logs = wb.create_sheet(title="Logs de Acesso") ws_users = wb.create_sheet(title="Cadastro Colaboradores") # Estilos e Cores (Paleta Corporativa) PRIMARY_DARK = "1E293B" HEADER_BG = "0F172A" CARD_BG = "F8FAFC" SUCCESS_BG = "DCFCE7" SUCCESS_FG = "166534" DANGER_BG = "FEE2E2" DANGER_FG = "991B1B" BORDER_COLOR = "CBD5E1" font_family = "Segoe UI" font_title = Font(name=font_family, size=16, bold=True, color="FFFFFF") font_header = Font(name=font_family, size=11, bold=True, color="FFFFFF") font_bold = Font(name=font_family, size=11, bold=True) font_regular = Font(name=font_family, size=10) fill_header = PatternFill( start_color=HEADER_BG, end_color=HEADER_BG, fill_type="solid" ) fill_card = PatternFill( start_color=CARD_BG, end_color=CARD_BG, fill_type="solid" ) fill_success = PatternFill( start_color=SUCCESS_BG, end_color=SUCCESS_BG, fill_type="solid" ) font_success = Font(name=font_family, size=10, bold=True, color=SUCCESS_FG) fill_danger = PatternFill( start_color=DANGER_BG, end_color=DANGER_BG, fill_type="solid" ) font_danger = Font(name=font_family, size=10, bold=True, color=DANGER_FG) thin_border = Border( left=Side(style="thin", color=BORDER_COLOR), right=Side(style="thin", color=BORDER_COLOR), top=Side(style="thin", color=BORDER_COLOR), bottom=Side(style="thin", color=BORDER_COLOR), ) # ---------------------------------------------------------- # ABA 1: Dashboard & Métricas # ---------------------------------------------------------- ws_dash.views.sheetView[0].showGridLines = True # Banner Principal ws_dash.merge_cells("A1:G2") banner_cell = ws_dash["A1"] banner_cell.value = ( " SISTEMA DE CONTROLE DE ACESSO AO COFRE DE TOXINAS - MINISTÉRIO DO MEIO AMBIENTE - AUDITORIA" ) banner_cell.font = font_title banner_cell.fill = PatternFill( start_color=PRIMARY_DARK, end_color=PRIMARY_DARK, fill_type="solid" ) banner_cell.alignment = Alignment(vertical="center", horizontal="left") # Cards de KPIs kpis = [ ( "A4:B5", "TOTAL DE ACESSOS", "=COUNTA('Logs de Acesso'!A2:A100)", "Total registrado", ), ( "C4:D5", "ACESSOS PERMITIDOS", '=COUNTIF(\'Logs de Acesso\'!D2:D100, "PERMITIDO")', "Acessos autorizados", ), ( "E4:F5", "TENTATIVAS NEGADAS", '=COUNTIF(\'Logs de Acesso\'!D2:D100, "NEGADO")', "Tentativas por período", ), ] for range_str, title, formula, sub in kpis: ws_dash.merge_cells(range_str) top_left = range_str.split(":")[0] cell = ws_dash[top_left] cell.value = f"{title}\n\n{formula}" cell.font = font_bold cell.alignment = Alignment( wrap_text=True, horizontal="center", vertical="center" ) cell.fill = fill_card start_col, start_row = range_str.split(":")[0][0], int( range_str.split(":")[0][1] ) end_col, end_row = range_str.split(":")[1][0], int( range_str.split(":")[1][1] ) for r in range(start_row, end_row + 1): for c in
```

```
range(ord(start_col) - 64, ord(end_col) - 64 + 1): ws_dash.cell(row=r, column=c).border = thin_border # Tabela Resumo ws_dash["A7"] = "Resumo de Acessos por Compartimento do Cofre (Nível)" ws_dash["A7"].font = Font( name=font_family, size=13, bold=True, color=PRIMARY_DARK ) headers_summary = ["Compartimento (Nível)", "Acessos Permitidos", "Acessos Negados", "% Sucesso"] for col_num, h in enumerate(headers_summary, 1): c = ws_dash.cell(row=8, column=col_num) c.value = h c.font = font_header c.fill = fill_header c.alignment = Alignment(horizontal="center", vertical="center") c.border = thin_border setores = [ "Nível 1 – Permissão Geral", "Nível 2 – Diretor de Divisões Específicas", "Nível 3 – Ministro do Meio Ambiente", ] for idx, setor in enumerate(setores, start=9): ws_dash.cell(row=idx, column=1, value=setor).font = font_regular ws_dash.cell( row=idx, column=2, value=f'=COUNTIFS(\'Logs de Acesso\'!C:C, A{idx}, \'Logs de Acesso\'!D:D, "PERMITIDO")', ).font = font_regular ws_dash.cell( row=idx, column=3, value=f'=COUNTIFS(\'Logs de Acesso\'!C:C, A{idx}, \'Logs de Acesso\'!D:D, "NEGADO")', ).font = font_regular p_cell = ws_dash.cell( row=idx, column=4, value=f"=IF((B{idx}+C{idx})>0, B{idx}/(B{idx}+C{idx}), 0)", ) p_cell.font = font_regular p_cell.number_format = "0.0%" for col_num in range(1, 5): c = ws_dash.cell(row=idx, column=col_num) c.border = thin_border if col_num > 1: c.alignment = Alignment(horizontal="center") # ---------------------------------------------------------- # ABA 2: Logs de Acesso # ----------------------------------------------------------
ws_logs.views.sheetView[0].showGridLines = True headers_logs = [ "ID Log", "Data / Hora", "Compartimento Solicitado", "Status", "Colaborador Detectado", "Nível Biométrico (Match)", ] for col_num, h in enumerate(headers_logs, 1): c = ws_logs.cell(row=1, column=col_num) c.value = h c.font = font_header c.fill = fill_header c.alignment = Alignment(horizontal="center", vertical="center") c.border = thin_border sample_logs = [ ( 1001, "2026-08-27 08:14:22", "Nível 1 – Permissão Geral", "PERMITIDO", "Carlos Eduardo Silva", "98.4%", ), ( 1002, "2026-08-27 08:30:10", "Nível 2 – Diretor de Divisões Específicas", "PERMITIDO", "Ana Beatriz Souza", "96.1%", ), ( 1003, "2026-08-27 09:02:45", "Nível 3 – Ministro do Meio Ambiente", "NEGADO", "Carlos Eduardo Silva", "97.8%", ), ( 1004, "2026-08-27 09:15:00", "Nível 2 – Diretor de Divisões Específicas", "PERMITIDO", "Mariana Oliveira", "99.1%", ), ( 1005, "2026-08-27 10:22:18", "Nível 3 – Ministro do Meio Ambiente", "PERMITIDO", "Lucas Mendes", "94.5%", ), ( 1006, "2026-08-27 11:05:33", "Nível 1 – Permissão Geral", "NEGADO", "Desconhecido / Visitante", "42.0%", ), ] for row_idx, row_data in enumerate(sample_logs, start=2): for col_idx, val in enumerate(row_data, start=1): c = ws_logs.cell(row=row_idx, column=col_idx, value=val) c.font = font_regular c.border = thin_border c.alignment = Alignment(horizontal="center") if col_idx == 4: if val == "PERMITIDO": c.fill = fill_success c.font = font_success else: c.fill = fill_danger c.font =
```

```
font_danger # ------------------------------------------------------------ # ABA 3: Cadastro Colaboradores # ------------------------------------------------------------ ws_users.views.sheetView[0].showGridLines = True headers_users = [ "ID", "Nome Completo", "Cargo / Função", "Divisão", "Nível de Acesso", "Status Cadastro", "Data Cadastramento", ] for col_num, h in enumerate(headers_users, 1): c = ws_users.cell(row=1, column=col_num) c.value = h c.font = font_header c.fill = fill_header c.alignment = Alignment(horizontal="center", vertical="center") c.border = thin_border sample_users = [ ( 1, "Carlos Eduardo Silva", "Analista de Segurança", "Divisão de Toxicologia Ambiental", "Nível 1 – Permissão Geral", "Ativo", "2026-01-15", ), ( 2, "Ana Beatriz Souza", "Diretora", "Divisão de Emergências Químicas", "Nível 2 – Diretor de Divisões Específicas", "Ativo", "2026-02-01", ), ( 3, "Mariana Oliveira", "Ministra do Meio Ambiente", "Gabinete do Ministro", "Nível 3 – Ministro do Meio Ambiente", "Ativo", "2026-01-10", ), ( 4, "Lucas Mendes", "Diretor", "Divisão de Fiscalização e Segurança Química", "Nível 2 – Diretor de Divisões Específicas", "Ativo", "2026-03-20", ), ] for row_idx, row_data in enumerate(sample_users, start=2): for col_idx, val in enumerate(row_data, start=1): c = ws_users.cell(row=row_idx, column=col_idx, value=val) c.font = font_regular c.border = thin_border if col_idx in [1, 5, 6]: c.alignment = Alignment(horizontal="center") # Ajuste automático das colunas for ws in [ws_dash, ws_logs, ws_users]: for col in ws.columns: max_len = 0 col_letter = get_column_letter(col[0].column) for cell in col: if cell.coordinate in [ "A1", "B1", "C1", "D1", "E1", "F1", "G1", "A2", "B2", "C2", "D2", "E2", "F2", "G2", ]: continue if cell.value: max_len = max(max_len, len(str(cell.value))) ws.column_dimensions[col_letter].width = max(max_len + 4, 12) # Salvar planilha file_name = "Relatorio_Controle_Acesso_Facial.xlsx" wb.save(file_name) print(f"Planilha '{file_name}' gerada com sucesso!")
```